HFS-VAE: A HIERARCHICAL FEATURE SPLITTING VARIATIONAL AUTOENCODER FOR INTERPRETABLE AND ROBUST NETWORK INTRUSION DETECTION

A Project Report submitted in the partial fulfillment of the Requirements for the award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shaik. Mohammad Thaheer (22471A05D1)
Shaik. Mannepalli Mastan Vali (22471A05C9)
Derangula. Durga Rao (22471A0585)

Under the esteemed guidance of

M. Suresh M.Tech.,
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NARASARAOPETA ENGINEERING COLLEGE: NARASAROPET

(AUTONOMOUS)

Accredited by NAAC with A+ Grade an ISO 9001:2015 Certified
Approved by AICTE, New Delhi, Permanently Affiliated to JNTUK, Kakinada
KOTAPPAKONDA ROAD, YALAMANDA VILLAGE, NARASARAOPET- 522601

2025– 2026

NARASARAOPETA ENGINEERING COLLEGE

(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project that is entitled with the name "HFS-VAE: A HIERARCHICAL FEATURE-SPLITTING VARIATIONAL AUTO ENCODER FOR INTERPRETABLE AND ROBUST NETWORK INTRUSION DETECTION" is a bonafide work done by the team Shaik. Mohammad Thaheer (22471A05D1) , Shaik. Mannepalli Mastan Vali (22471A05C9), Derangula. Durga Rao (22471A0585) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

PROJECT GUIDE
M. Suresh M.Tech.,
Assistant Professor

PROJECT CO-ORDINATOR
Dr. Sireesha Moturi, B.Tech., M.Tech ., Ph.D.,
Associate Professor

HEAD OF THE DEPARTMENT
Dr. S. N. Tirumala Rao, M.Tech., Ph.D.,
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We declare that this project work titled "**HFS-VAE: A HIERARCHICAL FEATURESPLITTING VARIATIONALAUTOENCODER FOR INTERPRETABLE VARIATIONAL AUTOENCODER FOR INTERPRETABLE AND ROBUST NETWORK INTRUSION DETECTION**" is composed by ourselves that the work contain here is our own except where explicitly stated otherwise in the text and that this work has been submitted for any other degree or professional qualification except as specified.

.

By

Shaik. Mohammad Thaheer (22471A05D1)

Shaik. Mannepalli Mastan Vali (22471A05C9)

Derangula. Durga Rao (22471A0585)

ACKNOWLEDGEMENT

We wish to express our thanks to various personalities who are responsible for the completion of the project. We are extremely thankful to our beloved chairman**,Sri M. V. Koteswara Rao, B.Sc.,** who took keen interest in us in every effort throughout this course. We owe our sincere gratitude to our beloved principal**, Dr. S. Venkateswarlu, M.Tech,Ph.D.,** for showing his kind attention and valuable guidance throughout the course.

We express our deep-felt gratitude towards **Dr. S. N. Tirumala Rao, M.Tech., Ph.D.,** HOD of the CSE department, and also to our guide**, M. Suresh, M.Tech.,** Assistant Professor of the CSE department, whose valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

We extend our sincere thanks to **Dr. Sireesha Moturi, B.Tech, M.Tech.,Ph.D.,** Associate professor & Project Coordinator of the project, for extending her encouragement. Their profound knowledge and willingness have been a constant source of inspiration for us throughout this project work.

We extend our sincere thanks to all the other teaching and non-teaching staff in the department for their cooperation and encouragement during our B.Tech. degree.

We have no words to acknowledge the warm affection, constant inspiration, and encouragement that we received from us parents.

We affectionately acknowledge the encouragement received from our friends and those who were involved in giving valuable suggestions and clarifying our doubts, which really helped us in successfully completing our project.

By

Shaik. Mohammad Thaheer (22471A05D1)

Shaik. Mannepalli Mastan Vali (22471A05C9)

Derangula. Durga Rao (22471A0585)

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

INSTITUTE VISION AND MISSION

INSTITUTION VISION

To emerge as a Centre of excellence in technical education with a blend of effective student centric teaching learning practices as well as research for the transformation of lives and community.

INSTITUTION MISSION

M1: Provide the best class infra-structure to explore the field of engineering and research

M2: Build a passionate and a determined team of faculty with student centric teaching, imbibing experiential, innovative skills

M3: Imbibe lifelong learning skills, entrepreneurial skills and ethical values in students for addressing societal problems

NARASARAOPETA ENGINEERING COLLEGE
(AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To become a center of excellence in nurturing the quality Computer Science & Engineering professionals embedded with software knowledge, aptitude for research and ethical values to cater to the needs of industry and society.

MISSION OF THE DEPARTMENT

The department of Computer Science and Engineering is committed to

M1: Mould the students to become Software Professionals, Researchers and Entrepreneurs by providing advanced laboratories.

M2: Impart high quality professional training to get expertize in modern software tools and technologies to cater to the real time requirements of the Industry.

M3: Inculcate team work and lifelong learning among students with a sense of societal and ethical responsibilities.

Program Specific Outcomes (PSO's)

PSO1: Apply mathematical and scientific skills in numerous areas of Computer Science and Engineering to design and develop software-based systems.

PSO2: Acquaint module knowledge on emerging trends of the modern era in Computer Science and Engineering

PSO3: Promote novel applications that meet the needs of entrepreneur, environmental and social issues.

Program Educational Objectives (PEO's)

The graduates of the programme are able to:

PEO1: Apply the knowledge of Mathematics, Science and Engineering fundamentals to identify and solve Computer Science and Engineering problems.

PEO2: Use various software tools and technologies to solve problems related to the academia, industry and society.

PEO3: Work with ethical and moral values in the multi-disciplinary teams and can communicate effectively among team members with continuous learning.

PEO4: Pursue higher studies and develop their career in software industry.

Program Outcomes

PO1: Engineering knowledge: Apply knowledge of mathematics, natural science,computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/development of solutions: Design creative solutions for complex
engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

PO4: Conduct investigations of complex problems: : Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: **Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: **Project management and finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: **Life-long learning :** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Project Course Outcomes (CO'S):

CO421.1: Analyse the System of Examinations and identify the problem.
CO421.2: Identify and classify the requirements.
CO421.3: Review the Related Literature
CO421.4: Design and Modularize the project
CO421.5: Construct, Integrate, Test and Implement the Project.
CO421.6: Prepare the project Documentation and present the Report using appropriate method.

Course Outcomes – Program Outcomes mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1		✓										✓		
C421.2	✓		✓		✓							✓		
C421.3				✓		✓	✓	✓				✓		
C421.4			✓			✓	✓	✓				✓	✓	
C421.5					✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
C421.6									✓	✓	✓	✓	✓	

Course Outcomes – Program Outcome correlation

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
C421.1	2	3										2		
C421.2			2		3							2		
C421.3				2		2	3	3				2		
C421.4			2			1	1	2				3	2	
C421.5					3	3	3	2	3	2	2	3	2	1
C421.6									3	2	1	2	3	

Note: The values in the above table represent the level of correlation between CO's and PO's:

1.Low level **2.** Medium level
3.Highlevel

Project mapping with various courses of Curriculum with Attained PO's:

Name of the course from which principles are Applied in this project	Description of the device	Attained PO
C2204.2,C22L3.2	Identification of the research problem and planning for the development of a deep learning-based intrusion detection system using HFS-VAE architecture.	PO1,PO3, PO8
CC421.1, C2204.3, C22L3.2	Requirement analysis and selection of hierarchical feature-splitting and variational autoencoder concepts for model design.	PO2, PO3,PO8
CC421.2, C2204.2, C22L3.3	Logical design of the system using UML diagrams, including data preprocessing, feature grouping, and latent feature fusion modules.	PO3, PO5, PO8,PO9
CC421.3, C2204.3,C22L3.2	Implementation, testing, and integration of the HFS-VAE model using CICIDS2017 and NSL-KDD datasets for intrusion detection.	PO1, PO5,PO8
CC421.4, C2204.4, C22L3.2	Preparation of project documentation covering all phases of development.	PO10,PO8
CC421.5, C2204.2, C22L3.3	Periodic presentation of progress, methodology, and outcomes during evaluations.	PO8,PO10, PO11
C2202.2,C2203.3,C1206.3, C3204.3, C4110.2	Implementation of the model using Python and deep learning frameworks on Google Colab for enterprise security applications. Tumor	PO5, PO7,PO8
C32SC4.3	Design of a conceptual dashboard for real-time network intrusion detection and visualization.	PO5, PO6 ,PO8

ABSTRACT

The complexity of modern networked environments has increased the risk of advanced cyber threats, making effective Network Intrusion Detection Systems (NIDS) essential. Deep learning has improved anomaly detection, but many models still lack robustness and interpretability. We propose the Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE), a novel architecture that partitions network features into semantic groups and encodes them through parallel branches, followed by classifier fusion. This design enhances transparency in the latent space and improves anomaly localization compared to conventional VAEs. Experiments on CICIDS2017 demonstrate an Average Precision of 0.8412 and an F1-score of 0.7463, while tests on NSL-KDD confirm adaptability with an AP of 0.9234 and an F1 of 0.7371. Robustness is validated through PCA-based latent visualizations, Maximum Likelihood Estimation (MLE), and boundary complexity analysis. Although recall remains lower than precision in some cases and evaluation is limited to CICIDS2017 and NSL-KDD, HFS-VAE provides a practical balance of interpretability and accuracy. Compared with prior approaches, it achieves competitive detection performance and superior anomaly separation, offering a structured and interpretable solution for enterprise-scale cybersecurity.

INDEX

S.NO	CONTENT	PAGE NO
1	INTRODUCTION	1
	1.1 MOTIVATION	2
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVE	4
2	LITERATURE SURVEY	5
	2.1 RELATED WORK	5
	2.2 APPLICATIONS OF DEEP LEARNING	7
3	SYSTEM ANALYSIS	8
	3.1 EXISTING SYSTEM	8
	3.1.1 DISADVANTAGES OF THE EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	9
	3.2.1 ADVANTAGES OF PROPOSED SYSTEM	11
	3.3 FEASIBILITY STUDY	12
	3.4 USING COCOMO MODEL	13
4	SYSTEM REQUIREMENTS	14
	4.1 SOFTWARE REQUIREMENTS	14
	4.2 REQUIREMENT ANALYSIS	14
	4.3 HARDWARE REQUIREMENTS	15
	4.4 SOFTWARE	16
	4.5 SOFTWARE DESCRIPTION	16
5	SYSTEM DESIGN	18
	5.1 SYSTEM ARCHITECTURE	18
	5.1.1 DATASET	19
	5.1.2 DATA PREPROCESSING	20
	5.1.3 FEATURE EXTRACTION	21
	5.1.4 MODEL BUILDING	22
	5.1.5 CLASSIFICATION	22
	5.2 MODULES	23
	5.2.1 DATASET LOADING MODULE	24
	5.2.2 DATA PREPROCESSING	24

	MODULE	
	5.2.3 FEATURE GROUPING MODULE	25
	5.2.4 HFS-VAE FEATURE LEARNING MODULE	26
	5.2.5 LATENT FEATURE EXTRACTION MODULE	26
	5.2.6 ATTACK CLASSIFI-CATION MODULE	27
	5.2.7 EVALUATION MODULE	27
	5.2.8 VISUALIZATION MODULE	28
	5.2.9 MODEL SAVING MODULE	28
	5.2.10 INFERENCE MODULE	28
	5.3 UML DIAGRAMS	29
	5.4 CONCLUSION	31
6	IMPLEMENTATION	32
	6.1 MODEL IMPLEMENTATION	32
	6.2 CODING	32
7	TESTING	64
	7.1 UNIT TESTING	64
	7.2 INTEGRATION TESTING	66
8	TEST CASES	70
9	RESULT ANALYSIS	71
10	OUTPUT SCREENS	74
11	CONCLUSION	77
12	FUTURE SCOPE	79
13	REFERENCES	80

LIST OF FIGURES

S.NO	CONTENT	PAGE NO
1	Figure 3.1Workflow of HFS-VAE Intrusion Detection System	10
2	Figure 3.2 System Architecture of the Proposed Model	11
3	Figure 5.1 System Architecture of the Proposed HFS-VAE Based Intrusion Detection System	19
4	Figure 5.2 Precision-Recall Curve of HFS-VAE + XGBoost	23
5	Figure 5.3 Latent Space Decision Boundary Visualization	23
6	Figure 5.4 Use Case Diagram for Intrusion Detection System	29
7	Figure 5.5 Activity Diagram of the Proposed Intrusion Detection System	30
8	Figure 5.6 Class Diagram of the Intrusion Detection System	31
9	Figure 7.1 Confusion Matrix	67
10	Figure 7.2 Precision-Recall Curve for Attack Detection	67
11	Figure 7.3 Model Accuracy Comparison Chart	69
12	Figure 8.1 Test Case 1 – Normal Traffic	70
13	Figure 8.2 Test Case 2 – Attack Traffic	70
14	Figure 9.1 Confusion Matrix of Proposed Intrusion Detection Model	72
15	Figure 9.2 Precision-Recall Curve for the Proposed Model	72
16	Figure 9.3 PCA Visualization of Latent Feature Space	73
17	Figure 10.1 Home Page Interface of the HFS-VAE Intrusion Detection System	74
18	Figure 10.2 Dataset Page of HFS-VAE Intrusion Detection System	74
19	Figure 10.3 Contact Us Page Interface of HFS-VAE Intrusion Detection System	75
20	Figure 10.4 Dashboard Page Interface of HFS-VAE Intrusion Detection System	75
21	Figure 10.5 Login Page Interface of HFS-VAE Intrusion Detection System	76
22	Figure10.6Register Page Interface of HFS-VAE Intrusion Detection System	76

1. INTRODUCTION

With the rapid growth of digital communication, cloud computing, and Internet-based services, modern computer networks have become increasingly complex and vulnerable to cyber threats. Organizations today rely heavily on interconnected systems to manage sensitive data, financial transactions, and critical infrastructure. However, this increased connectivity also exposes networks to a wide range of cyber attacks such as Distributed Denial of Service (DDoS), port scanning, botnets, infiltration attacks, and other malicious activities. Detecting and preventing these intrusions is therefore a critical challenge in modern cybersecurity.

Network Intrusion Detection Systems (NIDS) play an important role in monitoring network traffic and identifying suspicious or malicious behavior. Traditional intrusion detection approaches are generally categorized into **signature-based detection** and **anomaly-based detection**. Signature-based systems detect attacks by comparing network traffic patterns with previously known attack signatures. Although these systems are highly effective in identifying known threats, they struggle to detect **new or zero-day attacks** because such attacks do not yet have predefined signatures.

To address these limitations, anomaly-based detection methods have been developed. These methods learn the normal behavior of network traffic and detect deviations from that behavior as potential intrusions. Recently, **machine learning and deep learning techniques** have shown significant potential in improving the performance of intrusion detection systems. Among these techniques, **Autoencoders and Variational Autoencoders (VAEs)** have been widely used for anomaly detection because they can learn compressed representations of network traffic and identify abnormal patterns based on reconstruction errors or latent space distributions.

Despite these advantages, traditional VAE-based intrusion detection models still face several challenges. Many models lack **interpretability**, making it difficult for security analysts to understand how the model identifies anomalies. Additionally, deep learning models may be vulnerable to **adversarial perturbations**, where small changes in input data can cause incorrect predictions. Another limitation is that most models treat all network features equally, without considering the semantic relationships between different types of network traffic attributes.

To overcome these limitations, this project introduces a **Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE)** for network intrusion detection. The proposed approach divides network features into meaningful groups such as flow statistics, packet-level attributes, and protocol-related indicators. Each group is processed through separate encoder branches that learn specialized representations of the data. These latent representations are then combined to form a unified latent space, which improves the interpretability and robustness of the model.

Furthermore, the model integrates **distributional loss functions and classifier fusion techniques** to enhance anomaly detection performance. After learning the latent representations through the HFS-VAE architecture, a machine learning classifier such as **XGBoost** is applied to perform final classification of network traffic into benign or malicious categories. This hybrid approach leverages both deep learning representation learning and powerful gradient boosting classification.

The effectiveness of the proposed model is evaluated using benchmark datasets such as **CICIDS2017** and **NSL-KDD**, which contain diverse network traffic patterns and attack types. Performance is analyzed using several evaluation metrics including **Average Precision (AP), F1-score, AUC-ROC, and reconstruction loss**, along with visualization techniques such as **Principal Component Analysis (PCA)** for analyzing the learned latent space.

By combining hierarchical feature partitioning, variational autoencoder architectures, and hybrid classification methods, the proposed HFS-VAE framework aims to provide an **interpretable, robust, and scalable intrusion detection system** suitable for modern enterprise networks.

1.1 MOTIVATION

The increasing number of cyber attacks and the growing complexity of network infrastructures have created an urgent need for advanced intrusion detection systems. Traditional signature-based intrusion detection systems are limited in their ability to detect unknown or zero-day attacks, which continue to evolve rapidly in modern cyber environments.

Recent advances in deep learning have improved anomaly detection capabilities, but many existing models still suffer from limited interpretability and lack robustness against adversarial noise. In security-critical applications, it is important

not only to detect attacks accurately but also to understand how the detection decisions are made.

Another major challenge is that most intrusion detection models treat network traffic features as a single group of attributes, ignoring the underlying relationships between flow statistics, packet features, and protocol-level indicators. This can limit the model's ability to learn meaningful representations of network behavior.

Therefore, there is a strong motivation to design a system that can:

- Improve the interpretability of deep learning-based intrusion detection models
- Enhance robustness against adversarial or noisy network traffic
- Learn specialized representations from different groups of network features
- Provide accurate and scalable intrusion detection for enterprise networks

The proposed **HFS-VAE model** is motivated by these challenges and aims to address them through hierarchical feature splitting and hybrid deep learning techniques.

1.2 PROBLEM STATEMENT

Modern network environments generate large volumes of complex traffic data, making it difficult for traditional intrusion detection systems to accurately identify malicious activities. Existing signature-based systems are unable to detect new and unknown attacks, while many machine learning-based approaches lack transparency and robustness.

Although Variational Autoencoders have been successfully applied for anomaly detection, conventional VAE models treat all input features equally and do not exploit the semantic structure of network traffic data. This limitation reduces their ability to provide interpretable insights into network behavior and may lead to suboptimal detection performance.

Furthermore, deep learning-based intrusion detection models often suffer from issues such as class imbalance, adversarial vulnerability, and difficulty in identifying specific attack patterns within high-dimensional network traffic data.

Therefore, there is a need for an improved intrusion detection framework that can effectively learn structured representations of network traffic, provide interpretable results, and maintain robust detection performance across different datasets.

1.3 OBJECTIVES

The main objective of this project is to design and implement an advanced intrusion detection system based on a **Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE)** to improve anomaly detection in network traffic data.

The specific objectives of the proposed system are:

1. To develop a hierarchical feature-based deep learning model for network intrusion detection.
2. To partition network traffic features into meaningful groups to improve representation learning.
3. To design a multi-branch VAE architecture capable of learning specialized latent representations.
4. To integrate a hybrid classification mechanism using XGBoost for accurate attack detection.
5. To evaluate the performance of the proposed system using benchmark datasets such as CICIDS2017 and NSL-KDD.
6. To analyze the model's effectiveness using evaluation metrics such as Average Precision, F1-score, and AUC-ROC.
7. To visualize and interpret the learned latent representations using dimensionality reduction techniques.

2. LITERATURE SURVEY

2.1 RELATED WORK

The evolution of Intrusion Detection Systems (IDSs) has been significantly influenced by advances in deep learning. Early studies by *Shone et al*. (2018) [1] and *Vinayakumar et al*. (2019) [2] highlighted that although signature-based IDS techniques are effective for detecting known attack patterns, they fail to generalize to zero-day and previously unseen threats. To address these limitations, anomaly-based intrusion detection strategies were introduced, where models learn representations of normal network behavior and identify deviations as potential intrusions. A comprehensive review by *Pang et al*. (2021) [3] further emphasized the suitability of deep learning–based anomaly detection for complex and evolving network environments.

Autoencoders (AEs) and their variants have been widely employed in IDS for reconstruction-based anomaly detection. The seminal work by *Kingma and Welling* (2013) [4] introduced Variational Autoencoders (VAEs), which extend conventional autoencoders by learning probabilistic latent representations, thereby improving generalization and uncertainty modeling. Building upon this foundation, several studies explored robust VAE variants, including adversarial and sparse formulations, to enhance resilience against noisy and malicious traffic. In this direction, *He et al.* (2024) [6] proposed the NIDS-Vis framework, which emphasizes explainability and robustness through feature-space partitioning, latent-space visualization, adversarial boundary analysis, and robustness evaluation. To further enhance interpretability, dimensionality reduction techniques such as t-SNE proposed by van *der Maaten and Hinton* (2008) [9] and UMAP introduced by *McInnes et al.* (2018) [10] have been commonly adopted for visualizing learned latent embeddings.

The choice of dataset remains a critical factor in evaluating IDS performance. The UQ-IoT dataset introduced by *Marcelino et al*. (2021) [7] focuses on IoT network environments but suffers from limited attack diversity and class imbalance. Similarly, NSL-KDD analyzed by *Tavallaee et al.* (2009) [11], although an improvement over KDD'99, reflects outdated traffic patterns and lacks modern attack behaviors. In contrast, CICIDS2017 developed by *Sharafaldin et al.* (2018) [12] provides a more realistic enterprise-scale traffic environment with diverse attack

scenarios and balanced traffic distribution, making it a widely accepted benchmark for contemporary intrusion detection research.

To improve robustness and generalization on high-dimensional datasets, partitioned and grouped learning strategies have been proposed. *Chen and Li* (2021) [13] introduced grouped autoencoders that process semantically related feature subsets independently, allowing more effective representation learning. Hybrid detection pipelines that combine deep generative models with classical machine learning classifiers have also shown promising results. For instance, *Liu et al.* (2022) [14] demonstrated that combining latent embeddings with XGBoost classifiers improves detection accuracy and robustness. Complementary evaluation approaches include latent density estimation using Maximum Likelihood Estimation, as proposed by *Zong et al.* (2018) [15], and adversarial boundary complexity analysis investigated by *Wang et al.* (2021) [5].

Recent studies have further expanded IDS research in multiple directions. Mohamed (2023) [16] presented a comparative evaluation of VAE, AAE, and VAE-GAN models for network anomaly detection, highlighting trade-offs between stability and reconstruction capability. *Ren et al. (2023)* [17] proposed lightweight VAE-based IDS architectures tailored for IoT and resource-constrained environments. Hybrid deep learning and tree-based IDS models were explored by *Farhan et al*. (2025) [18], while adaptive intrusion detection using reinforcement learning was systematically reviewed by *Jamshidi et al.* (2025) [19]. Broader surveys focusing on scalability, efficiency, and interpretability of deep learning–based IDS were presented by *Zhang et al*. (2025) [20], emphasizing the growing need for explainable and deployable security solutions.

Building on these research directions, the proposed Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE) integrates hierarchical feature partitioning, modular latent learning, and explainability within a unified framework. Unlike prior approaches that primarily focus on detection accuracy, HFS-VAE emphasizes transparent latent-space analysis, anomaly localization, and robustness on enterprise-scale datasets, aligning with the increasing demand for interpretable and trustworthy IDS solutions in real-world cybersecurity deployments.

2.2 APPLICATIONS OF DEEP LEARNING

Deep learning plays a central role in the design and functioning of the HFS-VAE intrusion detection model, enabling the system to analyze large volumes of network traffic and accurately identify abnormal behaviour. The use of hierarchical feature partitioning, modular latent learning, and VAE-based reconstruction makes it suitable for several real-world cybersecurity applications.

One major application is enterprise network security, where organizations need continuous monitoring of incoming and outgoing traffic. The HFS-VAE model can automatically learn normal patterns within the enterprise network and detect deviations that indicate attacks such as DoS, DDoS, brute force, or infiltration attempts. This helps security teams respond faster and reduce the risk of system compromise. Another important application is in cloud and data-center environments, where large-scale traffic flows occur every second. Deep learning allows the system to handle high-dimensional data and identify complex attack signatures that traditional rule-based IDS solutions cannot detect. This makes the model suitable for cloud-hosted infrastructures, virtual networks, and API-based platforms.

The model can also be used in IoT and smart device networks, which generate continuous data but often lack strong security. The ability of deep learning to learn compact feature representations helps detect abnormal communication patterns in IoT devices, preventing botnet attacks, unauthorized access, and device-level exploitation. A further application lies in zeroday attack detection. Since HFS-VAE learns the normal behaviour of traffic rather than depending on fixed signatures, it can flag new, unseen, or evolving attack patterns. This makes the system highly effective for modern cybersecurity scenarios where threats are constantly changing.

Deep learning-based intrusion detection also has applications in automated security operations, where the model can support automated threat classification, visualization of latent patterns, and priority-based alerting. This reduces the workload on human analysts and enhances the efficiency of SOC (Security Operations Center) workflows. Overall, deep learning enables the HFS-VAE intrusion detection system to provide accurate, scalable, and adaptive security solutions suitable for enterprise networks.

3. SYSTEM ANALYSIS

System analysis is an important phase in the development of any software system. It involves understanding the problem domain, analyzing the limitations of existing approaches, and proposing an improved solution that addresses these limitations. In the context of network security, detecting cyber attacks in real-time is a major challenge due to the increasing complexity of network traffic and evolving attack strategies.

Traditional intrusion detection systems rely on signature-based detection methods which can only identify previously known attack patterns. However, modern cyber threats often include zero-day attacks and unknown malicious behaviors that cannot be easily detected using traditional methods. Therefore, intelligent and adaptive systems based on machine learning and deep learning are required to effectively identify both known and unknown network attacks.

This project focuses on designing an advanced intrusion detection system using a hybrid approach that combines deep learning with machine learning techniques. The system utilizes a **Hierarchical Feature Selection Variational Autoencoder (HFS-VAE)** to learn meaningful representations of network traffic and then applies the **XGBoost classifier** to perform accurate attack classification.

The proposed system is trained using the **CICIDS dataset**, which contains realistic network traffic data including both normal activities and multiple types of cyber attacks. Through proper preprocessing, feature extraction, and model training, the system aims to improve detection accuracy while reducing false alarms.

3.1 EXISTING SYSTEM

Existing intrusion detection systems mainly rely on traditional machine learning algorithms and signature-based detection techniques. These systems typically analyze predefined patterns of network traffic and compare them with known attack signatures stored in a database.

While these systems are effective for detecting previously identified attacks, they face several limitations when dealing with modern cyber threats. Attackers continuously modify their techniques, making it difficult for static signature-based systems to keep up with evolving attack patterns.

Some existing machine learning approaches such as Decision Trees, Support Vector Machines, and Random Forest classifiers have been used for network intrusion

detection. However, these models often rely heavily on manually engineered features and may struggle to capture complex relationships within high-dimensional network data.

Additionally, many traditional systems suffer from high false positive rates, which can lead to unnecessary alerts and increased workload for network administrators. These limitations highlight the need for more advanced approaches capable of learning deeper representations of network traffic patterns.

Another challenge in existing systems is the imbalance between normal and attack traffic within datasets. This imbalance can negatively impact classification performance and reduce the reliability of the detection system.

3.1.1 DISADVANTAGES OF EXISTING SYSTEM

The existing intrusion detection systems suffer from several limitations, which affect their effectiveness in modern network environments.

• Traditional systems depend heavily on predefined attack signatures, making them ineffective against unknown or zero-day attacks.

• Many machine learning models require extensive manual feature engineering, which may not capture complex patterns in network traffic data.

• High dimensional network datasets increase computational complexity and reduce model efficiency.

• Existing approaches often produce high false positive rates, causing unnecessary security alerts.

• Difficulty in detecting subtle anomalies within large volumes of network traffic.

• Limited capability to adapt to evolving cyber attack strategies.

These drawbacks indicate the necessity of developing an improved intrusion detection system that can automatically learn meaningful features and accurately detect both normal and malicious network activities.

3.2 PROPOSED SYSTEM

To overcome the limitations of existing systems, this project proposes a hybrid intrusion detection framework that integrates deep learning with advanced machine learning techniques.

The proposed system uses a **Hierarchical Feature Selection Variational Autoencoder (HFS-VAE)** to learn latent representations of network traffic data. The

VAE model helps in capturing complex relationships between features and reduces the dimensionality of the dataset while preserving important information.

The workflow of the proposed system begins with loading the CICIDS dataset and performing preprocessing operations such as data cleaning, normalization, and labeling. After preprocessing, the features are partitioned into different categories such as flow features, packet features, and protocol features.

The HFS-VAE model is trained using benign traffic data to learn the normal behavior of network traffic. Once the model is trained, it generates latent vectors that represent the underlying structure of the data.

These latent vectors are then used as input features for the **XGBoost classifier**, which performs the final classification between benign and attack traffic. The classifier is trained using labeled data to improve detection accuracy.

Finally, the performance of the system is evaluated using various evaluation metrics such as precision, recall, and average precision scores. Visualization techniques like PCA projection and decision boundary analysis are used to better understand the model's behavior.

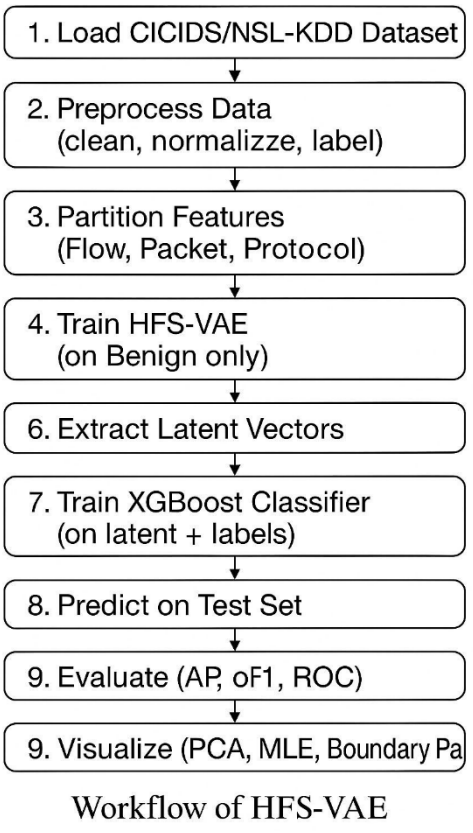

Figure 3.1 – Workflow of HFS-VAE Intrusion Detection System

The proposed architecture improves detection performance by combining the feature learning capability of deep learning with the classification power of gradient boosting algorithms. This hybrid approach enhances the ability of the system to detect complex and previously unseen attack patterns.

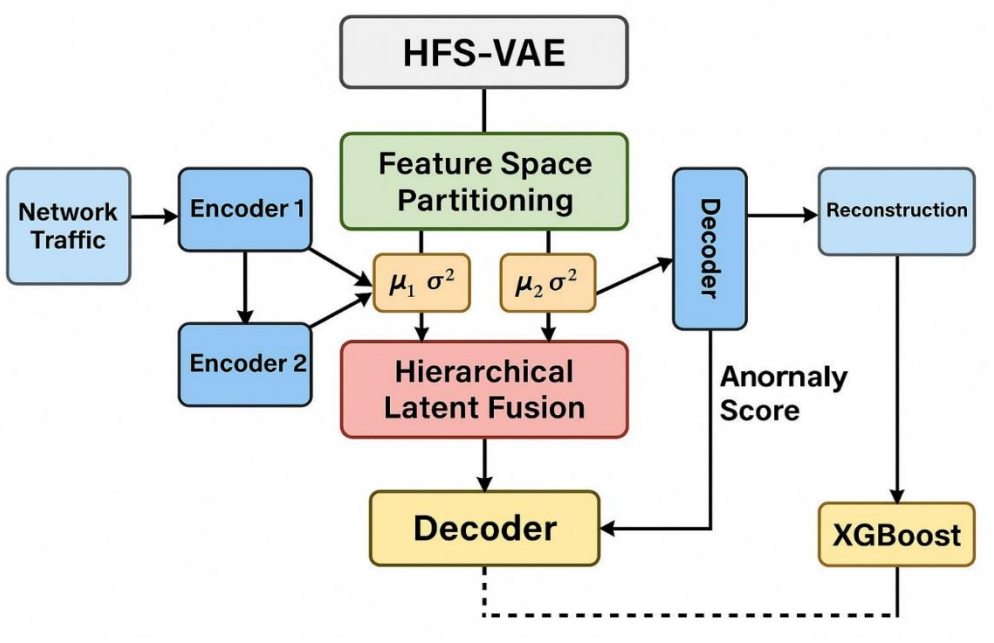

Figure 3.2 – System Architecture of the Proposed Model

3.2.1 ADVANTAGES OF THE PROPOSED SYSTEM

The proposed intrusion detection system provides several improvements over traditional network security solutions. By combining deep learning with machine learning techniques, the system achieves better detection accuracy and improved performance when analyzing complex network traffic patterns.

One of the major advantages of the proposed system is its ability to automatically learn meaningful feature representations from raw network data using the HFS-VAE model. This reduces the need for manual feature engineering and allows the system to capture complex relationships within the dataset.

Another advantage is the integration of the XGBoost classifier, which enhances classification performance by efficiently handling high-dimensional data and providing better generalization capability. The combination of latent feature extraction and

gradient boosting classification significantly improves the accuracy of attack detection. The proposed system is also capable of detecting both normal and malicious network activities with higher reliability. By learning patterns from benign traffic, the model can identify abnormal behaviors that may indicate potential cyber attacks.

Additionally, the system improves scalability and efficiency when processing large volumes of network traffic data. This makes it suitable for real-world network environments where data is continuously generated.

Key Advantages

• Improved detection accuracy for network intrusions
• Ability to detect unknown and zero-day attacks
• Reduced false positive rate compared to traditional systems
• Automatic feature extraction using deep learning models
• Efficient handling of high-dimensional network datasets
• Better scalability for large-scale network traffic analysis
• Integration of deep learning and machine learning for enhanced performance

3.3 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed system is practical, cost-effective, and technically viable for implementation. It helps determine whether the project can be successfully developed within the given constraints.

Technical Feasibility

The proposed system uses widely available technologies such as Python, machine learning libraries, and deep learning frameworks. These tools support efficient data processing and model training, making the system technically feasible to implement.

Operational Feasibility

The system is designed to assist network administrators in identifying suspicious activities in network traffic. By providing automated intrusion detection capabilities, it reduces manual monitoring efforts and improves operational efficiency.

Economic Feasibility

The system does not require expensive hardware or specialized infrastructure. The models can be trained using standard computing resources, making the solution economically feasible for organizations.

Schedule Feasibility

The development process follows a structured approach including dataset preparation, model development, testing, and evaluation. With proper planning and resource allocation, the project can be completed within the defined timeline.

3.4 USING COCOMO MODEL

The **Constructive Cost Model (COCOMO)** is used to estimate the effort, development time, and cost required for software development projects. It provides a systematic approach to estimate project resources based on the size and complexity of the software.

In this project, the COCOMO model helps estimate the development effort required for building the intrusion detection system. The model considers factors such as project size, complexity, and development environment.

According to the basic COCOMO model, the effort required for development can be estimated using the formula:

Effort = a × (KLOC)^b

Where:

KLOC = Thousands of lines of code

a and b = Constants based on project type

The estimated development time is calculated using:

Time = c × (Effort)^d

Where c and d are model constants.

Using these formulas, developers can estimate the approximate effort and time required to complete the project. This helps in proper planning and efficient allocation of resources.

4. SYSTEM REQUIREMENTS

System requirements describe the necessary hardware and software components required for the successful implementation of the proposed intrusion detection system. These requirements ensure that the system can effectively process network traffic data, train machine learning models, and perform intrusion detection tasks efficiently.

The proposed system relies on modern machine learning and deep learning frameworks to analyze network traffic patterns and detect potential cyber attacks. Proper system requirements ensure smooth development, model training, and evaluation processes.

The requirements are categorized into software requirements, hardware requirements, and detailed descriptions of the software tools used in the project.

4.1 SOFTWARE REQUIREMENTS

Software requirements define the programming environments, libraries, and tools necessary to develop and execute the intrusion detection system.

The system is implemented using the Python programming language due to its extensive support for data analysis, machine learning, and deep learning applications. Various Python libraries are used for data preprocessing, model development, and performance evaluation.

Required Software

• **Operating System** : Windows 10 or Linux

• **Programming Language** : Python

• **Development Environment** : Jupyter Notebook / VS Code

• **Machine Learning Libraries** : Scikit-learn, XGBoost

• **Deep Learning Framework** : TensorFlow / PyTorch

• **Data Processing Libraries** : NumPy, Pandas

• **Visualization Libraries** : Matplotlib, Seaborn

• **Browser** : Any modern browser (Chrome, Edge, Firefox)

These software tools provide efficient support for handling large network datasets and implementing machine learning algorithms for intrusion detection.

4.2 REQUIREMENT ANALYSIS

Requirement analysis is the process of identifying the functional and non-functional

requirements of the proposed system. It helps define how the system will operate and what functionalities it should provide.

Functional Requirements

Functional requirements describe the core operations that the system must perform.

• The system should load the CICIDS dataset containing network traffic data.
• It should preprocess the dataset by removing noise and handling missing values.
• The system must perform feature extraction and hierarchical feature selection.
• The HFS-VAE model should learn latent representations of normal network traffic.
• The system should classify network traffic using the XGBoost classifier.
• The model should detect both benign and malicious traffic patterns.
• The system should generate evaluation metrics such as precision, recall, and accuracy.

Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system.

• The system should process large network datasets efficiently.
• The model should maintain high detection accuracy.
• The system should minimize false positive detection rates.
• The implementation should be scalable for future network data analysis.
• The system should provide clear visualization of results for analysis.

Requirement analysis ensures that the system performs efficiently while meeting the objectives of network intrusion detection.

4.3 HARDWARE REQUIREMENTS

Hardware requirements specify the physical components needed to develop and execute the intrusion detection system.

Since the system primarily performs data processing and machine learning model training, moderate computing resources are sufficient for implementation.

Required Hardware

• **Processor** : Intel Core i5 or higher
• **RAM** : Minimum 8 GB
• **Storage** : Minimum 256 GB Hard Disk / SSD
• **GPU (Optional)** : NVIDIA GPU for faster deep learning training
• **System Architecture** : 64-bit system

These hardware components ensure training and evaluation of proposed model.

4.4 SOFTWARE

The system is implemented using various software tools and libraries that support machine learning, data processing, and visualization. These tools help in developing the intrusion detection system efficiently and enable effective analysis of network traffic data.

flexibility, and extensive ecosystem of machine learning libraries. Development environments such as Jupyter Notebook or Visual Studio Code are used for coding, debugging, and running experiments.

Machine learning frameworks such as Scikit-learn and XGBoost provide efficient algorithms for classification tasks, while deep learning frameworks like TensorFlow or PyTorch support the implementation of the Variational Autoencoder model.

4.5 SOFTWARE DESCRIPTION

This section provides a brief description of the software tools used in the project.

Python

Python is a high-level programming language widely used for data science, machine learning, and artificial intelligence applications. It offers numerous libraries that simplify data analysis and model development.

Jupyter Notebook

Jupyter Notebook is an interactive development environment that allows developers to write and execute Python code while visualizing outputs such as graphs and tables. It is commonly used for machine learning experimentation.

Scikit-learn

Scikit-learn is a powerful machine learning library in Python that provides tools for data preprocessing, model training, and performance evaluation. It supports various algorithms used for classification and data analysis.

XGBoost

XGBoost is an optimized gradient boosting algorithm designed for high performance and efficiency. It is widely used for classification and regression tasks and provides excellent performance on structured datasets.

TensorFlow / PyTorch

TensorFlow and PyTorch are deep learning frameworks used for building neural network models. In this project, they support the implementation of the Variational Autoencoder for learning latent representations of network traffic data.

Matplotlib and Seaborn

These libraries are used for data visualization and result analysis. They help generate plots such as confusion matrices, precision-recall curves, and other graphical representations of model performance.

The system requirements described in this chapter provide the necessary foundation for implementing the proposed intrusion detection system. Both hardware and software requirements have been identified to ensure that the system can efficiently process large network traffic datasets and support the implementation of machine learning and deep learning models.

The use of Python along with powerful libraries such as Scikit-learn, XGBoost, TensorFlow, and data visualization tools enables effective development and evaluation of the proposed model. The defined hardware configuration ensures that the system can perform data preprocessing, model training, and performance evaluation without significant computational limitations.

Overall, the specified requirements ensure that the proposed system can be developed, tested, and deployed efficiently, enabling accurate detection of network intrusions and improving cybersecurity monitoring capabilities.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

This project focuses on designing an intelligent **Network Intrusion Detection System (NIDS)** using a hybrid deep learning architecture that combines **Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE)** and **XGBoost classifier**. The goal of the system is to automatically detect malicious network traffic by learning compact latent representations of high-dimensional network flow features.

Traditional intrusion detection models rely on handcrafted feature engineering and shallow machine learning algorithms, which often struggle to handle large-scale network traffic datasets. To overcome these limitations, the proposed architecture integrates **deep generative representation learning** with **gradient boosting classification**. This hybrid approach enables the system to capture complex patterns in network traffic while maintaining high classification accuracy.

The system begins with network traffic records from the **CICIDS2017 dataset**, which contains labeled benign and malicious network flows. These flows consist of multiple statistical features such as packet length, flow duration, inter-arrival time, and protocol characteristics.

The overall architecture consists of the following major stages:
• Dataset acquisition and loading
• Data preprocessing and feature normalization
• Feature grouping and hierarchical encoding
• Latent representation learning using HFS-VAE
• Intrusion classification using XGBoost
• Evaluation and visualization of results

In the first stage, raw network traffic records are collected from the CICIDS2017 dataset. These records are cleaned, normalized, and transformed into numerical feature vectors suitable for deep learning models.

Next, the system divides the features into **multiple hierarchical groups**. Each group is processed through a **Variational Autoencoder encoder block**, which learns compressed representations of the data. These encoded representations are combined to form a **latent feature vector** representing each network flow.

The latent representation is then provided to an **XGBoost classifier**, which performs final classification between benign and attack traffic.
This architecture enables the model to capture both **local feature interactions** within feature groups and **global patterns** across the entire network traffic dataset.

The architecture of the proposed system is illustrated in Fig. 5.1.

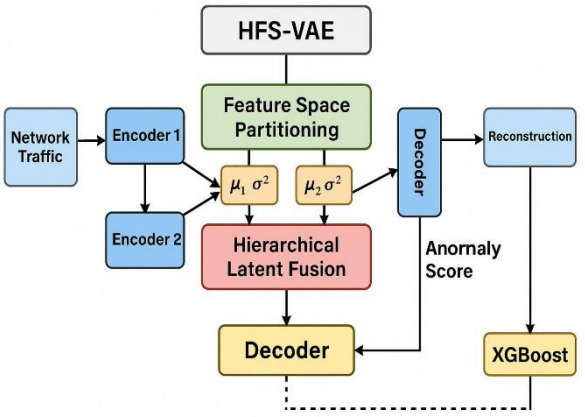

Fig 5.1 System Architecture of the Proposed HFS-VAE Based Intrusion Detection System

5.1.1 DATASET

The dataset used in this project is the CICIDS2017 dataset, which is a widely used benchmark dataset for network intrusion detection research. The dataset was created by the Canadian Institute for Cybersecurity and contains realistic network traffic that includes both benign activities and various types of cyber attacks.
The dataset consists of network flow records generated from simulated enterprise network environments. These records capture detailed information about network communication such as packet counts, byte statistics, protocol types, flow duration, and timing information.
The CICIDS2017 dataset includes several types of attacks, such as:

• Distributed Denial of Service (DDoS)
• Brute Force attacks
• Web attacks
• Botnet activity
• Port scanning
• Infiltration attacks

Each network flow record is labeled as either BENIGN or ATTACK, enabling supervised machine learning approaches to be applied.

Dataset Characteristics:
• Total Network Flows: ~2.8 Million
• Number of Features: 78 network traffic features
• Traffic Type: Benign and malicious network flows
• Attack Categories: Multiple real-world attack scenarios
• Data Format: CSV files containing flow statistics

During preprocessing, multiple CSV files from the dataset are merged into a single dataset for training and evaluation.

Feature	Description
Dataset Name	CICIDS2017
Total Samples	~2.8 Million
Number of Features	78
Attack Types	DDoS, Brute Force, Botnet, Web Attack
Data Format	CSV
Application	Network Intrusion Detection

The dataset provides a realistic representation of network traffic behavior, making it suitable for evaluating the performance of intrusion detection systems.

5.1.2 DATA PREPROCESSING

Data preprocessing is a critical step in building an effective intrusion detection system because raw network traffic data often contains missing values, inconsistent formats, and irrelevant attributes.

The preprocessing pipeline applied in this project includes several steps to ensure data quality and improve model performance.

Data Cleaning

Network flow records containing missing values, infinite values, or corrupted entries are removed from the dataset. This prevents instability during model training.

Feature Selection

Certain attributes such as:
• Flow ID
• Source IP
• Destination IP
• Timestamp

are removed because they do not contribute directly to intrusion detection and may introduce noise into the model.
Feature Scaling
All numerical features are normalized using MinMaxScaler, which scales feature values between 0 and 1.
The normalization process can be expressed as:

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

where:

X = original feature value
Xmin = minimum value of the feature
Xmax = maximum value of the feature
Normalization ensures that all features contribute equally during model training.
Train-Test Split

The dataset is divided into:
• Training data (80%)
• Validation data (20%)

Only benign samples are used to train the VAE model, allowing it to learn normal network behavior. Attack samples are used during testing to evaluate anomaly detection performance.

5.1.3 FEATURE EXTRACTION

Feature extraction is performed using a Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE).
Instead of processing all features together, the system divides the input features into three groups:

Group 1 – First 20 features
Group 2 – Next 20 features
Group 3 – Remaining features

Each feature group is passed through an independent VAE encoder.
A Variational Autoencoder consists of:
• Encoder network
• Latent representation
• Decoder network
The encoder learns two parameters:

$$z_{mean}, z_{logvar}$$

The latent vector is sampled using the reparameterization trick:

$$z = z_{mean} + e^{0.5 z_{logvar}} * \epsilon$$

where ε is a random noise sampled from a normal distribution.
The latent vectors from the three feature groups are concatenated to form a final latent representation of the network flow.
This approach allows the system to learn structured representations of network behavior while reducing dimensionality.

5.1.4 MODEL BUILDING

The proposed model integrates HFS-VAE and XGBoost to create a hybrid intrusion detection system.

Latent Representation Learning

The VAE model learns compressed representations of network traffic data by minimizing reconstruction loss and KL divergence.
The VAE loss function is defined as:

$$Loss = Reconstruction\ \ Loss + KL\ \ Divergence$$

Reconstruction loss measures the difference between the original input and reconstructed output.

KL divergence ensures the latent variables follow a normal distribution.

XGBoost Classifier

After latent representations are generated, they are passed to an XGBoost classifier.
XGBoost is a gradient boosting algorithm that builds multiple decision trees sequentially to improve classification accuracy.

Advantages of XGBoost include:

• High performance on structured data
• Ability to capture complex feature interactions
• Robustness against overfitting
The classifier outputs the probability of each network flow being malicious or benign.

5.1.5 CLASSIFICATION

The final stage of the system is the classification of network traffic.
The latent feature vectors generated by the HFS-VAE encoder are used as input to the XGBoost classifier.

The classifier predicts the class label:
• 0 → Benign traffic
• 1 → Malicious attack

The classification output can be mathematically expressed as:

$$Y = f(Z)$$

where:
Z = latent feature representation
f = XGBoost classification function
Y = predicted class label

The performance of the model is evaluated using several metrics including:
• Precision
• Recall
• F1 Score
• ROC-AUC
• Average Precision (AP)

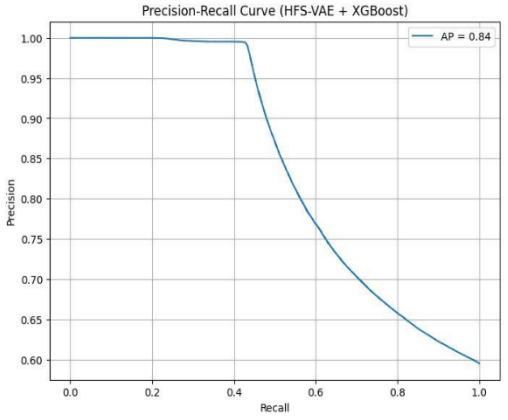

Fig 5.2 Precision Recall Curve of HFS-VAE + XGBoost

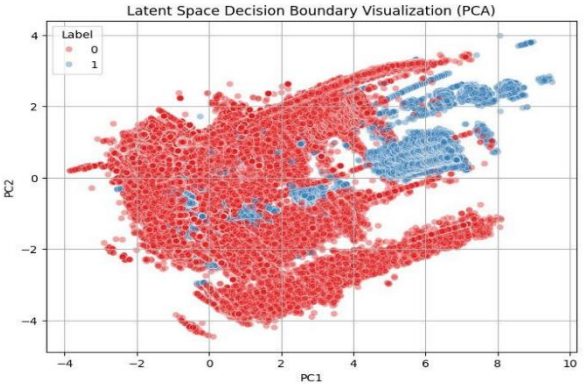

Fig 5.3 Latent Space Decision Boundary Visualization

5.2 MODULES

In software engineering, a module is a self-contained functional unit that performs a specific operation within a system. Dividing a complex system into modules improves maintainability, scalability, and clarity of design.
For the proposed Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE) based Network Intrusion Detection System, the system is organized into several functional modules. Each module handles a specific stage such as data loading, preprocessing, feature learning, classification, visualization, and evaluation.
This modular architecture ensures that the system can efficiently detect malicious network activities while maintaining flexibility for future improvements.

5.2.1. Dataset Loading Module

This module is responsible for loading the CICIDS2017 dataset, which contains labeled network traffic records representing both benign and attack activities.
The dataset consists of multiple CSV files representing different network traffic scenarios such as brute force attacks, DDoS attacks, and infiltration attempts. The module combines all CSV files into a single unified dataset for processing.
Functions of this module
• Reads multiple CSV files from the dataset folder
• Combines them into a single dataset
• Displays dataset structure and preview

Sample Code

```
from glob import glob
import pandas as pd
import os

folder_path = "/content/drive/MyDrive/Nids/NIDS-Vis-master/TrafficLabelling"
csv_files = glob(os.path.join(folder_path, "*.csv"))

dataframes = []
for f in csv_files:
    df = pd.read_csv(f, low_memory=False, encoding='ISO-8859-1')
    dataframes.append(df)

raw_df = pd.concat(dataframes, ignore_index=True)
print("Dataset Shape:", raw_df.shape)
```

5.2.2. Data Preprocessing Module

Raw network traffic data often contains missing values, redundant fields, and non-numerical attributes. This module prepares the dataset for machine learning by cleaning and transforming the data.
Preprocessing tasks
• Removing infinite and missing values
• Dropping non-relevant features such as IP addresses and timestamps
• Converting attack labels into numerical format
• Scaling features using MinMax normalization
Normalization ensures that all feature values fall within the same range, improving model training stability.
Feature Normalization Formula

$$X' = X - X_{min} / X_{max} - X_{min}$$

Where:
- X**= original feature value**
- X_{min}**= minimum value of feature**
- X_{max}**= maximum value of feature**

Sample Code

```
from sklearn.preprocessing import MinMaxScaler

labels = raw_df[' Label'].apply(lambda x: 0 if x == 'BENIGN' else 1)

df = raw_df.replace([np.inf, -np.inf], np.nan).dropna()

non_features = ['Flow ID',' Source IP',' Destination IP',' Timestamp',' Label']
df.drop(columns=[col for col in non_features if col in df.columns], inplace=True)

scaler = MinMaxScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
```

5.2.3. Feature Grouping Module

The HFS-VAE model introduces Hierarchical Feature Splitting, where the input features are divided into multiple groups to capture different characteristics of network traffic.

Instead of feeding all features to a single encoder, the system splits them into three hierarchical feature groups:

- Flow-based features
- Packet-based features
- Protocol-related features

Each group is processed by a separate encoder inside the HFS-VAE architecture.

Sample Code

```
group_1 = df_scaled.columns[:20]
group_2 = df_scaled.columns[20:40]
group_3 = df_scaled.columns[40:]
```

This design improves representation learning by allowing the model to learn specialized patterns for different feature categories.

5.2.4. HFS-VAE Feature Learning Module

This module implements the Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE).
The VAE learns compressed latent representations of benign network traffic patterns. Since it is trained only on normal traffic, any deviation from learned patterns can indicate malicious behavior.

The model contains:

• Three parallel encoders (one for each feature group)
• A latent space representation layer
• A decoder network that reconstructs input features

Variational Autoencoder Loss Function

The training objective combines reconstruction loss and Kullback–Leibler divergence:

$$\boldsymbol{L = Lreconstruction + KL(q(z \mid x) \mid\mid p(z))}$$

Where:

• $\boldsymbol{L_{reconstruction}}$**measures reconstruction error**

• $\boldsymbol{KL}$**regularizes latent space distribution**

Sample Code

```
inp = Input(shape=(input_dim,))
h = layers.Dense(16, activation='relu')(inp)

z_mean = layers.Dense(latent_dim)(h)
z_log_var = layers.Dense(latent_dim)(h)
```

The trained encoder produces latent vectors representing network behavior patterns.

5.2.5. Latent Feature Extraction Module

After training the HFS-VAE, the encoder is used to transform network traffic samples into low-dimensional latent representations.
These latent vectors summarize the important behavioral characteristics of network traffic.

Sample Code

```
encoder_model = Model(inputs=[in1, in2, in3], outputs=z)

Z_all = encoder_model.predict([X1_all, X2_all, X3_all])
```

These latent vectors are then used as input for the classification stage.

5.2.6. Attack Classification Module

The classification module uses XGBoost, a powerful gradient boosting algorithm, to classify network traffic into benign or attack categories.
XGBoost works by constructing multiple decision trees sequentially and combining them to improve prediction accuracy.
Advantages of XGBoost
• High performance on structured datasets
• Handles large datasets efficiently
• Robust against overfitting

Sample Code

```
from xgboost import XGBClassifier

clf = XGBClassifier(n_estimators=150)
clf.fit(x_train_cls, y_train_cls)
```

The classifier outputs predicted labels and probabilities for each network traffic sample.

5.2.7. Evaluation Module

The performance of the intrusion detection system is evaluated using several metrics:
• **Average Precision (AP)**
• **F1 Score**
• **ROC-AUC Score**
• **Detection Rate**
These metrics provide a comprehensive evaluation of classification performance.

Sample Code

```
from sklearn.metrics import average_precision_score, roc_auc_score

ap = average_precision_score(y_test, y_pred_prob)
roc_auc = roc_auc_score(y_test, y_pred_prob)
```

5.2.8. Visualization Module

Visualization helps analyze the behavior of the trained model and interpret its predictions.

The system generates several visual outputs including:
• **Confusion Matrix** – Shows classification accuracy
• **Precision-Recall Curve** – Evaluates classification performance
• **Latent Space Visualization (PCA)** – Shows clustering of traffic samples
• **MLE Density Histogram** – Shows local density estimation
These visualizations help understand how the model separates benign and malicious traffic.

Sample Code

```
plt.plot(recall, precision)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
```

5.2.9. Model Saving Module

After training, the system saves the trained components for future use.
Saved files include:
• HFS-VAE model
• MinMax scaler
• Feature list
• XGBoost classifier

Sample Code

```
import joblib
hfs_vae.save("hfs_vae_model.h5")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(clf, "xgb_model.pkl")
```

These files allow the system to perform real-time intrusion detection without retraining.

5.2.10. Inference Module

The inference module allows the trained system to analyze **new network traffic data** and detect potential intrusions.
The process includes:
1. Loading trained models
2. Preprocessing incoming traffic
3. Extracting latent features
4. Classifying traffic as benign or attack

This enables the system to function as a **real-time network security monitoring tool**.

5.3 UML Diagrams

Unified Modeling Language (UML) diagrams are used to visually represent the structure and behavior of the proposed Intrusion Detection System. These diagrams help in understanding how different components of the system interact with each other and how the data flows within the system. In this project, UML diagrams are used to describe the interaction between the user, the intrusion detection model, and the dataset processing modules.

The UML diagrams provide a clear representation of system functionality and improve the understanding of system design before implementation. They help in identifying system requirements, workflows, and communication between system components.

5.3.1 Use Case Diagram

The Use Case Diagram represents the interaction between the user and the intrusion detection system. It shows the major functionalities provided by the system such as uploading the dataset, preprocessing the data, training the machine learning model, detecting network attacks, and displaying the results.

The user interacts with the system by providing the dataset and initiating the training process. The system processes the dataset, performs feature extraction and dimensionality reduction, and then applies the machine learning algorithm to detect intrusions. The results are finally displayed to the user in the form of classification outputs and evaluation metrics.

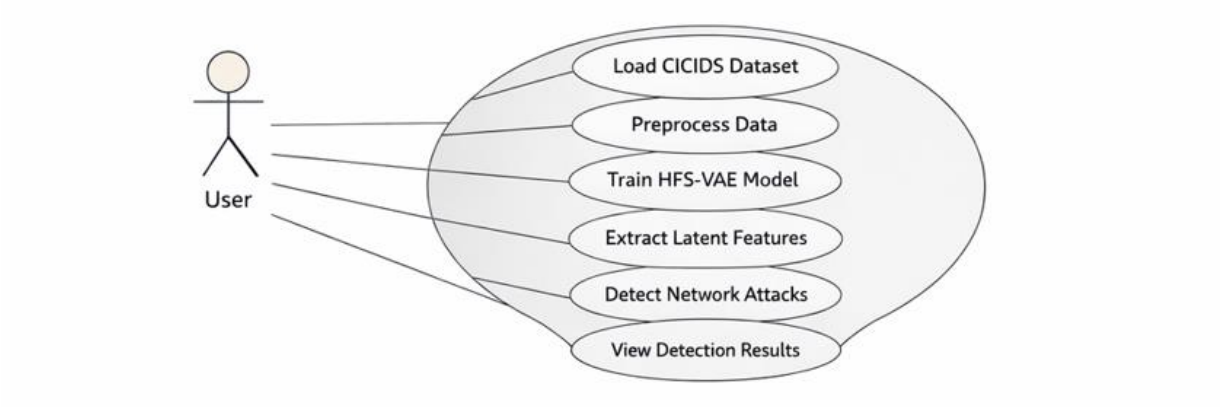

Figure 5.3 – Use Case Diagram for Intrusion Detection System

5.3.2 Activity Diagram

The Activity Diagram describes the workflow of the proposed intrusion detection system. It illustrates the sequence of activities performed during the detection process, starting from data input to final attack classification.

Initially, the dataset is uploaded to the system. The system then performs preprocessing steps such as data cleaning, normalization, and feature selection. After preprocessing, the dimensionality reduction technique is applied to reduce redundant features. The processed data is then passed to the machine learning model for training and testing. Finally, the model predicts whether the network traffic is normal or malicious and generates evaluation results.

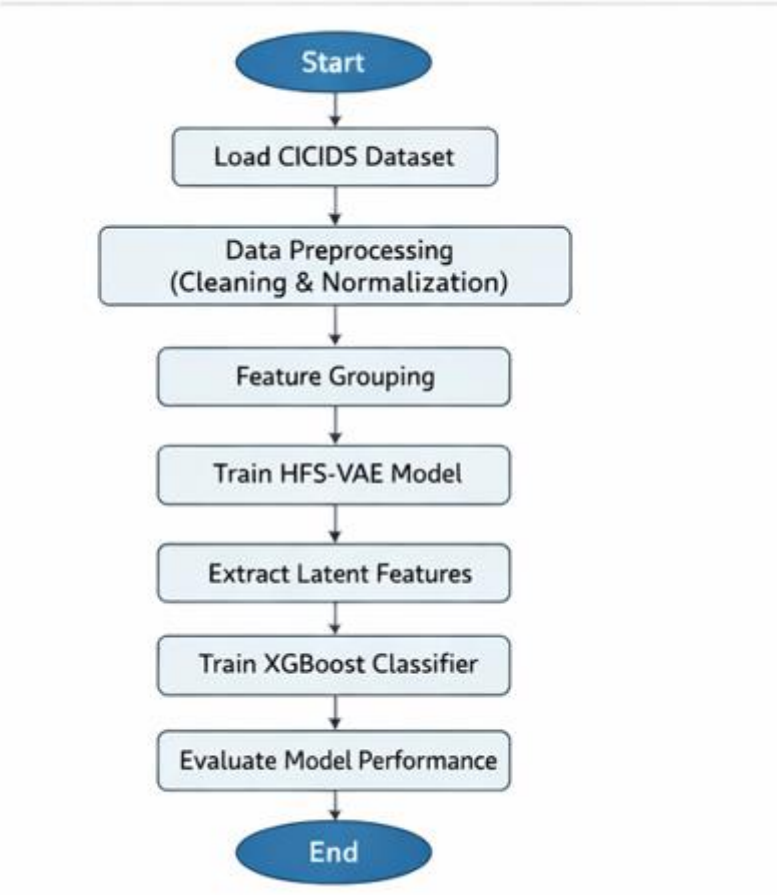

Figure 5.4 – Activity Diagram of the Proposed Intrusion Detection System

5.3.3 Sequence Diagram

The Sequence Diagram illustrates the interaction between different system components over time. It shows how the user request is processed by the system and how the system components communicate with each other to complete the intrusion detection task.
First, the user uploads the dataset and initiates the detection process. The dataset is passed to the preprocessing module, where data cleaning and feature extraction are performed. The processed data is then forwarded to the machine learning model, which analyzes the traffic patterns and classifies them into normal or attack categories. The results are then sent to the output module and displayed to the user.

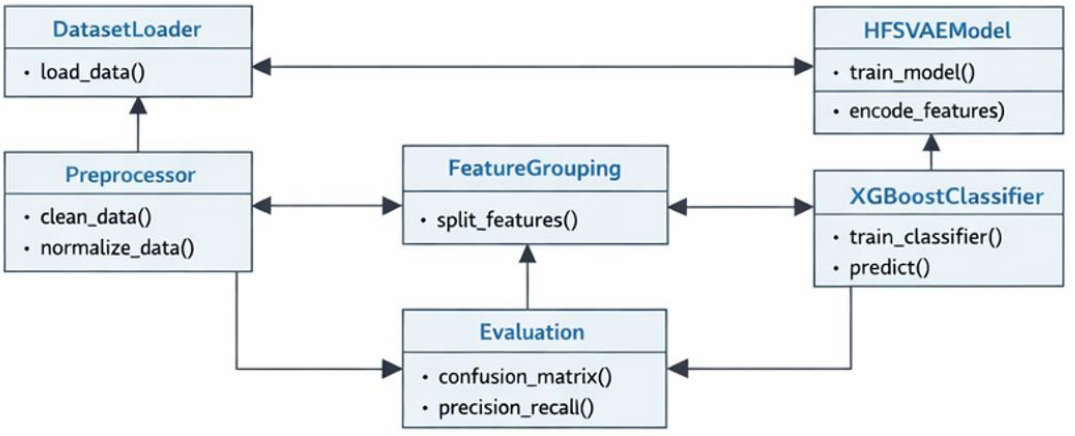

Figure 5.5 – Class Diagram the Intrusion Detection System

5.4 Conclusion

The system design of the proposed HFS-VAE based Network Intrusion Detection System presents a structured framework for detecting malicious network traffic using deep learning and visualization techniques. The architecture integrates data preprocessing, hierarchical feature extraction, latent representation learning using Variational Autoencoders, and classification through an XGBoost model. Each module is designed to perform a specific function while maintaining smooth interaction with other components, ensuring modularity and scalability.
The UML diagrams further illustrate the interactions between system components and the overall workflow from dataset input to final attack classification. By combining deep feature learning with visualization techniques such as PCA and density estimation, the proposed system provides both accurate intrusion detection and interpretable results. This modular design ensures efficient training, reliable prediction, and easier future enhancements for advanced cybersecurity applications.

6.IMPLIMENTATION

6.1 Model Implementation

The implementation phase focuses on developing the proposed intrusion detection system using deep learning and machine learning techniques. The system is implemented in Python using Google Colab, which provides GPU support for efficient model training. The main libraries used include NumPy, Pandas, Scikit-learn, TensorFlow/Keras, Matplotlib, and XGBoost.

The dataset used for implementation is the CICIDS2017 dataset, which contains both normal and malicious network traffic records. Multiple CSV files from the dataset are combined to create a single dataset for training and testing.

Before training the model, preprocessing steps are performed to clean the data. Missing values and infinite values are removed, and irrelevant attributes such as Flow ID, Source IP, Destination IP, and Timestamp are dropped. The attack labels are converted into numerical format where 0 represents benign traffic and 1 represents attack traffic. Feature values are then normalized using Min-Max scaling to improve model performance.

The proposed system uses a Hierarchical Feature Splitting Variational Autoencoder (HFS-VAE) to learn compressed latent representations of network traffic. The input features are divided into three groups, and each group is processed through separate encoder layers. The latent representations obtained from the encoder are combined to form a shared feature space.

After training the VAE model, the latent vectors are extracted and used as input features for the XGBoost classifier. The classifier learns to distinguish between benign and malicious traffic patterns. The performance of the model is evaluated using metrics such as Precision, Recall, F1-score, Average Precision, and ROC-AUC.

6.2 Coding

The coding part of the system is divided into multiple modules including dataset loading, preprocessing, model training, classification, and evaluation. The VAE model is built using TensorFlow/Keras, while the classification stage uses the XGBoost algorithm.
The trained models and preprocessing components are saved for future use, including:

- **hfs_vae_model.h5** – trained VAE model
- **scaler.pkl** – data normalization scaler
- **features.pkl** – feature list used for training
- **xgb_model.pkl** – trained XGBoost classifier

These saved models allow the system to perform intrusion detection on new network traffic data without retraining the model.

Front End Connection Code:

Home.js

```
import React from "react";
import "./Home.css";
export default function Home() {
  return (
    <div className="home-container">
      {/* HERO SECTION */}
      <section className="hero-section fade-in">
        <div>
          <h1>HFS-VAE</h1>
          <h2>
            Hierarchical Feature-Splitting Variational Autoencoder
          </h2>
          <p>
          A next-generation <strong>Network Intrusion Detection System
          </strong> engineered to deliver <strong>robust, interpretable, and
            enterprise-scale anomaly detection</strong> using structured deep
            learning and hierarchical latent modeling.
          </p>
        </div>
      </section>
      {/* ABOUT */}
      <section className="about-section grid-2 slide-left">
        <div>
          <h3>About the Project</h3>
```

```
    <p>
      HFS-VAE is a research-oriented cybersecurity framework developed to
       address critical limitations in traditional intrusion detection
      systems such as poor interpretability, weak adversarial robustness, and
      limited scalability.
    </p>

    <p>
      Unlike conventional deep learning IDS models that operate as black-box
      systems, HFS-VAE introduces structured latent learning that allows security
      analysts to visualize, interpret, and localize anomalies within network
      traffic.
    </p>

    <p>
      Network traffic features are divided into semantic groups such as
      <strong> Flow Statistics</strong>,
      <strong> Packet Metrics</strong>, and
      <strong> Protocol Indicators</strong>, enabling modular representation
      learning and improved anomaly explainability.
    </p>
  </div>

  <div className="highlight-card slide-right">
    <h3>Key Contributions</h3>
    <ul>
      <li>Hierarchical Feature Splitting Architecture</li>
      <li>Parallel Variational Encoder Branches</li>
      <li>Latent Space Fusion & Disentanglement</li>
      <li>XGBoost-based Classifier Fusion</li>
      <li>Distributional Loss for Latent Alignment</li>
      <li>Adversarial Robustness Regularization</li>
      <li>Visualization-Driven Interpretability</li>
    </ul>
  </div>
</section>
```

```
{/* METHODOLOGY */}
<section className="methodology-section fade-in">
  <h2 className="section-title">Core Methodology</h2>

  <div className="grid-3">
    <div className="info-card">
      <h4>Feature Partitioning</h4>
      <p>
        Traffic attributes are grouped into semantic clusters allowing
        specialized encoders to learn domain-specific representations.
      </p>
    </div>

    <div className="info-card">
      <h4>Distributional Loss Function</h4>
      <p>
        Aligns latent embeddings with Gaussian priors to enhance anomaly
        separation and density modeling.
      </p>
    </div>

    <div className="info-card">
      <h4>Latent Fusion</h4>
      <p>
        Multi-branch embeddings are fused into a unified latent vector for
        robust intrusion classification.
      </p>
    </div>
  </div>
</section>

{/* ARCHITECTURE */}
<section className="architecture-section grid-2 slide-left">
  <div>
    <h3>System Architecture</h3>
```

```
    <p>
      The HFS-VAE architecture consists of parallel encoder branches, each trained
      on a specific feature subset. The encoded latent vectors are concatenated and
      passed through a shared decoder for
      reconstruction.
    </p>

    <p>
      A hybrid detection pipeline is formed by integrating an
      <strong> XGBoost classifier</strong> on top of the fused latent embeddings,
      enabling supervised anomaly detection while preserving unsupervised
      representation learning.
    </p>
  </div>

  <div className="dataset-card slide-right">
    <h3>Robustness Mechanisms</h3>
    <ul>
      <li>Smoothness Regularization</li>
      <li>Latent Perturbation Stability</li>
      <li>Boundary Complexity Analysis</li>
      <li>Adversarial Noise Resistance</li>
    </ul>
  </div>
</section>

{/* DATASETS */}
<section className="dataset-section grid-2 slide-left">
  <div>
    <h3>Datasets Used</h3>
    <ul>
      <li>
        <strong>CICIDS2017</strong> – Modern enterprise attack traffic
        including DDoS, Botnet, PortScan, and Infiltration.
      </li>
      <li>
```

```
      <strong>NSL-KDD</strong> – Benchmark dataset for legacy
      intrusion detection evaluation.
    </li>
  </ul>
  <p>
    These datasets ensure validation across both contemporary and
    traditional network environments.
  </p>
</div>

<div className="dataset-card slide-right">
  <h3>Evaluation Metrics</h3>
  <ul>
    <li>Average Precision (AP)</li>
    <li>F1-Score</li>
    <li>AUC-ROC</li>
    <li>Reconstruction Loss</li>
    <li>MLE Density Scoring</li>
    <li>Latent Boundary Analysis</li>
  </ul>
</div>
</section>

{/* RESULTS */}
<section className="results-section fade-in">
  <h2 className="section-title">Performance Highlights</h2>

  <div className="grid-3">
    <div class Name="stat-card">
      <h3>0.8412</h3>
      <p>Average Precision</p>
    </div>

    <div class Name="stat-card">
      <h3>0.7463</h3>
```

```
      <p>F1-Score</p>
    </div>

    <div className="stat-card">
      <h3>0.88</h3>
      <p>AUC-ROC</p>
    </div>
  </div>

  <p className="results-text">
    Experimental evaluation demonstrates strong anomaly separation,
    extremely high precision, and interpretable latent clustering,
    validating HFS-VAE as a practical enterprise cybersecurity solution.
  </p>
</section>

{/* FUTURE WORK */}
<section className="future-section fade-in">
  <h2 className="section-title">Future Enhancements</h2>

  <div className="grid-3">
    <div className="info-card">
      <h4>Streaming IDS</h4>
      <p>Extend detection to real-time network streams.</p>
    </div>

    <div className="info-card">
      <h4>IoT Deployment</h4>
      <p>Optimize model for edge and IoT environments.</p>
    </div>

    <div className="info-card">
      <h4>Attack Categorization</h4>
      <p>Enable fine-grained multi-class intrusion labeling.</p>
    </div>
  </div>
```

```
      </section>

      {/* FOOTER */}
      <footer className="footer">
          <p>© 2026 HFS-VAE Research Project • Intelligent
          Cybersecurity Analytics</p>
      </footer>

    </div>
  );
}
```

Dataset.js

```
import React from "react";
import "./Dataset.css";

export default function Dataset() {

  const downloadFile = (filePath) => {
    const fileURL = window.location.origin + filePath;
    const link = document.createElement("a");
    link.href = fileURL;
    link.download = filePath.split("/").pop();
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };

  return (
    <div className="page-container">
      <h2>Dataset Information</h2>

      <p>
```

```
  Our Intrusion Detection System is trained and evaluated using high-
  quality, real-world network traffic datasets. These datasets contain
  millions of flow records, multiple attack categories and detailed
  statistical features required for deep-learning based anomaly detection
  using <strong>HFS-VAE</strong>.
</p>

{/* WHY DATASETS */}
<h3>🔬 Why These Datasets?</h3>
<p>
  Our HFS-VAE intrusion detection model is evaluated using two
  complementary datasets to ensure both <b>real-world relevance</b> and
  <b>benchmark comparability</b>.
</p>

<ul>
  <li>
    <strong>CIC-IDS 2017 — Modern Enterprise Traffic</strong><br />
    Contains realistic enterprise network behaviour with modern attacks
    including DDoS, Botnet, Web attacks, PortScan and Infiltration
    traffic.
  </li>

  <li>
    <strong>NSL-KDD — Benchmark Dataset</strong><br /> Used to
    validate generalization ability and compare results with prior intrusion
    detection research.
  </li>
</ul>

{/* PERFORMANCE */}
<h3>📊 Model Performance</h3>
<div  className="results-grid">
```

```
  <div className="result-card">
    <h4>CICIDS2017 Results</h4>
    <p><b>Average Precision:</b> 0.8412</p>
    <p><b>F1 Score:</b> 0.7463</p>
    <p>Strong detection of modern enterprise attacks.</p>
  </div>

  <div className="result-card">
    <h4>NSL-KDD Results</h4>
    <p><b>Average Precision:</b> 0.9234</p>
    <p><b>F1 Score:</b> 0.7371</p>
    <p>Excellent cross-dataset generalization.</p>
  </div>
</div>

{/* ADVANCED EVALUATION */}
<h3>☐ Advanced Evaluation</h3>
<ul>
  <li>PCA latent-space visualization for anomaly separation</li>
  <li>Maximum Likelihood Estimation (MLE) density scoring</li>
  <li>Decision boundary complexity analysis</li>
  <li>Hierarchical feature-splitting for interpretability</li>
</ul>

{/* OFFICIAL LINK */}
<h3>🔗 Official Dataset Source</h3>
<a
  className="dataset-link"
  href="https://www.unb.ca/cic/datasets/ids-2017.html"
  target="_blank"
  rel="noopener noreferrer"
>
  👉 Visit CIC-IDS2017 Official Website
```

```
</a>

{/* DOWNLOAD CICIDS */}
<h3>t Download CICIDS2017 CSV Files</h3>
<div className="download-grid">
  <button onClick={() => downloadFile("/datasets/cicids/Monday-WorkingHours.pcap_ISCX.csv")}>Monday Working Hours</button>
  <button onClick={() => downloadFile("/datasets/cicids/Tuesday-WorkingHours.pcap_ISCX(1).csv")}>Tuesday Working Hours</button>
  <button onClick={() => downloadFile("/datasets/cicids/Wednesday-workingHours.pcap_ISCX.csv")}>Wednesday Working Hours</button>
  <button onClick={() => downloadFile("/datasets/cicids/Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv")}>Thursday Web Attacks</button>
  <button onClick={() => downloadFile("/datasets/cicids/Thursday-WorkingHours-Afternoon-Infilteration.pcap_ISCX.csv")}>Thursday Infilteration</button>
  <button onClick={() => downloadFile("/datasets/cicids/Friday-WorkingHours-Morning.pcap_ISCX.csv")}>Friday Morning</button>
  <button onClick={() => downloadFile("/datasets/cicids/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv")}>Friday PortScan</button>
  <button onClick={() => downloadFile("/datasets/cicids/Friday-WorkingHours-Afternoon-DDoS.pcap_ISCX(1).csv")}>Friday DDoS</button>
</div>

{/* NSL KDD */}
<h3>t Download NSL-KDD Dataset</h3>
<div className="download-grid">
  <button onClick={() => downloadFile("/datasets/nslkdd/NSL_KDD.csv")}>Download NSL_KDD.csv</button>
</div>

{/* CUSTOM DATASETS */}
<h3>📌 Custom Dataset Support</h3>
```

```
      <p>
        Users can upload their own traffic CSV files via the Dashboard for
        real-time intrusion detection. The system supports datasets with
        network flow features such as IPs, ports, packet statistics and
        protocol attributes.
      </p>
    </div>
  );
}
```

DashBoard.js

```
import React from
"react"; import { Pie,
Bar } from "react-
chartjs-2"; import {
Chart as ChartJS,
ArcElement,
Tooltip,
  Legend,
  CategoryScale,

LinearSc
ale,
BarElem
ent,

import "./Dashboard.css";

// ✅ Register the chart components manually
ChartJS.regi
ster(
ArcElement,
Tooltip,
Legend,
CategoryScale,
```

```
LinearSc
ale,
BarElem
ent,

const Dashboard =
({ result }) => { if
(!result) return null;

  const { total_flows, normal, attacks } = result;

  const
pieData = {
labels:



["Normal",
"Attack"],
datasets: [
    {       data: [normal, attacks],
backgroundColor: ["#38bdf8",
"#ef4444"],
hoverBackgroundColor: ["#60a5fa",
"#f87171"],
    },
]
,
};
  const barData = { labels:
["Total Flows", "Normal",
"Attacks"], datasets: [
    {
      label:        "Flow
 Counts",              data:
[total_flows,     normal,
attacks],
      backgroundColor: ["#a78bfa", "#34d399", "#f87171"],
    },
```

```
  return (
    <div  className="dashboard-container">
      <div  className="summary-cards">
        <div className="card total">
          <h3>Total Flows</h3>
          <p>{total_flows}</p>
        </div>
        <div className="card normal">
          <h3>Normal</h3>
          <p>{normal}</p>

            </div>
        <div className="card attacks">
          <h3>Attacks</h3>
          <p>{attacks}</p>


      <div className="charts">
        <div className="chart-box">
          <h3>Traffic Distribution</h3>
          <Pie data={pieData} />
        </div>

        <div className="chart-box">
          <h3>Flow Overview</h3>
          <Bar data={barData} />
        </div>
      </div>
    </div>
  );
}; export
default
Dashboa
rd;
```

ContactUs.js

```
import React, { useState } from "react";
import "./ContactUs.css";

export default function ContactUs() {
  const [form, setForm] = useState({ name: "", email: "", message: "" });

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = (e) => {
    e.preventDefault();
    alert("📨 Message sent successfully!");
    setForm({ name: "", email: "", message: "" });
  };

  return (
    <div className="page-container">

      {/* ================= PROJECT INFO ============= */}
      <h2>Research & Contact</h2>
      <p className="center-text">
        This platform demonstrates our research on an advanced deep-
        learning
        Intrusion Detection System based on the <b>HFS-VAE</b>
        architecture.
      </p>

      <div className="info-grid">

        <div className="info-card">
          <h3>🎯 Research Objective</h3>
          <p>
            Our goal is to build an intelligent and interpretable network
            intrusion detection system capable of identifying modern cyber
            attacks in real-time enterprise environments.
          </p>
```

```
  </div>

  <div className="info-card">
    <h3>🔬 Research Contributions</h3>
    <ul>
      <li>Hierarchical Feature-Splitting VAE</li>
      <li>Latent-space anomaly detection</li>
      <li>MLE-based density scoring</li>
      <li>XGBoost classifier fusion</li>
      <li>Interactive web deployment</li>
    </ul>
  </div>

  <div className="info-card">
    <h3>🏢 Project Domain</h3>
    <p>
      Artificial Intelligence • Cyber Security • Deep Learning •

      Network Traffic Analysis • Explainable AI.
    </p>
  </div>

</div>


{/* ================= TEAM ================= */}
<h2 style={{marginTop:"80px"}}>Research Team</h2>

<div className="team-grid">

  <div className="team-card">
    <img src="/team/member1.png" alt="Researcher"/>
    <h3>Researcher 1</h3>
    <p>Deep Learning & IDS Research</p>
    <span>HFS-VAE Model Development</span>
  </div>
```

```
  <div className="team-card">
    <img src="/team/member2.jpeg" alt="Researcher"/>
    <h3>Researcher 2</h3>
    <p>Machine Learning & Data Engineering</p>
    <span>Dataset Processing & Training</span>
  </div>

  <div className="team-card">
    <img src="/team/member3.jpeg" alt="Researcher"/>
    <h3>Researcher 3</h3>
    <p>Cyber Security Analyst</p>
    <span>Evaluation & Testing</span>
  </div>

  <div className="team-card">
    <img src="/team/member4.jpeg" alt="Researcher"/>
    <h3>Researcher 4</h3>
    <p>Frontend & Deployment</p>
    <span>Web Application Development</span>
  </div>

</div>

{/* ======= CONTACT INFO ======== */}
<h2 style={{marginTop:"80px"}}>Get In Touch</h2>


<div className="contact-cards">

  <div className="contact-card">
    <h4>Email</h4>
    <p>shaikthaheer752@gmail.com</p>
  </div>

  <div className="contact-card">
    <h4>Project Type</h4>
    <p>Academic Research Project</p>
  </div>
```

```
      <div className="contact-card">
        <h4>Focus Area</h4>
        <p>AI Powered Intrusion Detection</p>
      </div>
    </div>

    {/* ================= FORM ================= */}
    <form className="contact-form" onSubmit={handleSubmit}>
      <input
        type="text"
        name="name"
        placeholder="Your Name"
        value={form.name}
        onChange={handleChange}
        required
      />

      <input
        type="email"
        name="email"
        placeholder="Your Email"
        value={form.email}
        onChange={handleChange}
        required
      />
      <textarea
        name="message"
        placeholder="Your Message"
        value={form.message}
        onChange={handleChange}

        required
      />

      <button type="submit">Send Message</button>
    </form>

  </div>
);
```

Login.js

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import "./Login.css";

const Login = () => {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");

  const handleLogin = () => {
    const storedEmail = localStorage.getItem("userEmail");
    const storedPassword = localStorage.getItem("userPassword");

    const normalizedEmail = email.trim().toLowerCase();

    if (!email || !password) {
      setError("Please enter email and password");
      return;
    }

    if (
      normalizedEmail === storedEmail &&
      password === storedPassword
    ) {
      setError("");
      alert("Login successful!");
      navigate("/home");

    } else {
      setError("Invalid email or password");
    }
  };

  return (
    <div className="login-page">

      <div className="login-wrapper">
```

```
{/* LOGIN CARD */}
<div className="login-card">

  <h2>HFS-VAE Portal Login</h2>
  <p className="sub-text">
    Access the Intrusion Detection Dashboard
  </p>

  {error && <div className="error-msg">{error}</div>}

  <div className="input-group">
    <label>Email Address</label>
    <input
      type="email"
      placeholder="Enter your registered email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
  </div>

  <div className="input-group">
    <label>Password</label>
    <div className="password-box">
      <input
        type={showPassword ? "text" : "password"}
        placeholder="Enter your password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <span
        className="toggle"
        onClick={() => setShowPassword(!showPassword)}
      >
        {showPassword ? "Hide" : "Show"}
      </span>
    /div>
  </div>

  <button className="login-btn" onClick={handleLogin}>
    Login to Dashboard
  </button>

</div>
```

```
        {/* SIDE WELCOME PANEL */}
        <div className="welcome-panel">
          <h1>Welcome Back</h1>
          <p>
            Monitor cyber threats, visualize anomalies, and explore
            HFS-VAE intrusion detection analytics.
          </p>

          <button onClick={() => navigate("/signup")}>
            Create Account
          </button>
        </div>

      </div>
    </div>
  );
};

export default Login;
```

SignUp.js

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import "./Login.css"; // Reusing same CSS theme

const Signup = () => {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);
  const [error, setError] = useState("");
```

```
const handleSignup = () => {
  if (!email || !password || !confirmPassword) {
    setError("All fields are required");
    return;
  }

  if (password.length < 6) {
    setError("Password must be at least 6 characters");
    return;
  }

  if (password !== confirmPassword) {
    setError("Passwords do not match");
    return;
  }

  const normalizedEmail = email.trim().toLowerCase();

  // Store credentials
  localStorage.setItem("userEmail", normalizedEmail);
  localStorage.setItem("userPassword", password);

  setError("");
  alert("Signup successful! Please login.");
  navigate("/login");
};

return (
  <div className="login-page">

    <div className="login-wrapper">

      {/* SIGNUP CARD */}
      <div className="login-card">

        <h2>Create HFS-VAE Account</h2>
        <p className="sub-text">
          Register to access intrusion detection analytics
        </p>

        {error && <div className="error-msg">{error}</div>}

        {/* EMAIL */}
        <div className="input-group">
```

```
  <label>Email Address</label>
  <input>
    type="email"
    placeholder="Enter your email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
</div>

{/* PASSWORD */}
<div className="input-group">
  <label>Password</label>
  <div className="password-box">
    <input
      type={showPassword ? "text" : "password"}
      placeholder="Create password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
    />
    <span
      className="toggle"
      onClick={() => setShowPassword(!showPassword)}
    >
      {showPassword ? "Hide" : "Show"}
    </span>
  </div>
</div>

{/* CONFIRM PASSWORD */}
<div className="input-group">
  <label>Confirm Password</label>
  <div className="password-box">
    <input
      type={showConfirm ? "text" : "password"}
      placeholder="Re-enter password"
      value={confirmPassword}
      onChange={(e) => setConfirmPassword(e.target.value)}
    />
    <span
      className="toggle"
      onClick={() => setShowConfirm(!showConfirm)}
    >
      {showConfirm ? "Hide" : "Show"}
    </span>
  </div>
```

```
          </div>

          <button className="login-btn" onClick={handleSignup}>
            Create Account
          </button>

        </div>

        {/* SIDE PANEL */}
        <div className="welcome-panel">
          <h1>Join HFS-VAE</h1>
          <p>
            Create an account to explore anomaly detection,
            visualize cyber threats, and monitor enterprise
            network security insights.
          </p>

          <button onClick={() => navigate("/login")}>
            Already Registered? Login
          </button>
        </div>

      </div>
    </div>
  );
};

export default Signup;
```

BACKEND

APP.PY

```
#
==========================================================
# 🚀 MAX SPEED HFS-VAE CICIDS2017 BACKEND (FINAL + EVALUATION)
# ==========================================================
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["OMP_NUM_THREADS"] = "8"
os.environ["TF_NUM_INTRAOP_THREADS"] = "8"
os.environ["TF_NUM_INTEROP_THREADS"] = "8"
from flask import Flask, request, jsonify
from flask_cors import CORS
import pandas as pd
import numpy as np
import time, traceback, joblib
import tensorflow as tf
from tensorflow.keras.models import load_model, Model
from tensorflow.keras import backend as K
tf.config.optimizer.set_jit(True)
```

```
tf.config.threading.set_intra_op_parallelism_threads(8)
tf.config.threading.set_inter_op_parallelism_threads(8)

app = Flask(__name__)
CORS(app)


UPLOAD_FOLDER = "uploads"
PREDICT_FOLDER = "predictions"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(PREDICT_FOLDER,  exist_ok=True)


# ============================================================
# Custom Sampling Layer
# ============================================================
def sampling(args):
    z_mean, z_log_var = args
    epsilon = K.random_normal(shape=K.shape(z_mean))
    return z_mean + K.exp(0.5 * z_log_var) * epsilon


# ============================================================
# Load Models
# ============================================================
print("🚀 Loading models...")
```

```
hfs_vae = load_model("hfs_vae_model.h5",
                     custom_objects={"sampling": sampling},
                     compile=False)

encoder = Model(inputs=hfs_vae.inputs,
                outputs=hfs_vae.layers[-3].output)

scaler = joblib.load("scaler.pkl")
FEATURES = joblib.load("features.pkl")
xgb_model = joblib.load("xgb_model.pkl")

input_shapes = [int(inp.shape[-1]) for inp in hfs_vae.inputs]
print("✅ Models Loaded")

# ============================================================
# ⚡ ULTRA FAST CSV PREDICTION (UPDATED)
# ============================================================
def predict_csv(file_path):

    chunk_size = 80000
    batch_size = 8192
```

```
total_flows = total_attacks = total_normal = 0
output_chunks = []

print("⚡ Prediction started...")

reader = pd.read_csv(file_path, chunksize=chunk_size,
                     engine="c", encoding="ISO-8859-1",
                     low_memory=False)

for chunk in reader:

    # 🔷 Keep ORIGINAL CICIDS data safe (contains Label column)
    original_chunk = chunk.copy()

    # 🔷 Extract only model features
    model_input = chunk[FEATURES]
    arr = model_input.to_numpy(dtype=np.float32, copy=False)

    arr[np.isinf(arr)] = 0
    arr[np.isnan(arr)] = 0
    np.clip(arr, -1e6, 1e6, out=arr)

    arr = scaler.transform(arr)
```

```
splits = np.split(arr, np.cumsum(input_shapes)[:-1], axis=1)

latent = encoder.predict(splits, batch_size=batch_size, verbose=0)
scores = xgb_model.predict_proba(latent)[:, 1]
labels = (scores > 0.10).astype(np.int8)

total_flows += len(labels)
total_attacks += labels.sum()
total_normal += len(labels) - labels.sum()

# ========================================================
# ☆ GET REAL CLASS FROM CICIDS FILE
# ========================================================
label_col = [c for c in original_chunk.columns if "label" in c.lower()][0]
real_class = np.where(original_chunk[label_col] == "BENIGN", "BENIGN",
"ATTACK")

predicted_class = np.where(labels == 1, "ATTACK", "BENIGN")

# ========================================================
# ☆ DETECT FALSE POSITIVES ONLY
# BENIGN predicted as ATTACK
# ========================================================
```

```
    false_pred = np.where(
        (real_class == "BENIGN") & (predicted_class == "ATTACK"),
        "False Prediction", ""
    )

    # ========================================================
    # ☆ ADD NEW COLUMNS TO ORIGINAL DATA
    # ========================================================
    original_chunk["Attack_Score"] = scores
    original_chunk["Class"] = real_class
    original_chunk["Predicted_Label"] = labels
    original_chunk["Predicted_Class"] = predicted_class
    original_chunk["False_Prediction"] = false_pred

    output_chunks.append(original_chunk)
    print(f"⚡ {total_flows} flows processed")

# ========================================================
# Save CSV
# ========================================================
output_df = pd.concat(output_chunks, ignore_index=True)
output_file = os.path.join(PREDICT_FOLDER,
                           "predicted_" + os.path.basename(file_path))
```

```
    output_df.to_csv(output_file, index=False)


    print("🚀 Prediction finished!")


    return {
        "total_flows": int(total_flows),
        "normal": int(total_normal),
        "attacks": int(total_attacks),
        "output_csv": output_file
    }


# ============================================================
# API
# ============================================================
@app.route("/predict_csv", methods=["POST"])
def predict_from_csv():
    file_path = None
    start_time = time.time()


    try:
        file = request.files["file"]
        file_path = os.path.join(UPLOAD_FOLDER, file.filename)
        file.save(file_path)
```

```
        result = predict_csv(file_path)
        result["processing_time_sec"] = round(time.time() - start_time, 2)
        return jsonify(result)

    except Exception as e:
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500

    finally:
        if file_path and os.path.exists(file_path):
            os.remove(file_path)

# ============================================================
if __name___== "__main__":
    app.run(debug=True)
```

7. TESTING

Testing is a critical stage in the development of the **Network Intrusion Detection System (NIDS)** designed for detecting malicious activities in network traffic. This phase ensures that every component of the system — including **data preprocessing, feature extraction, dimensionality reduction using PCA, deep learning model training, and visualization modules** — works correctly under different conditions.

The main objective of testing is to verify:

• Functional correctness of the detection system
• Accuracy of attack classification
• Stability of the machine learning pipeline
• Reliability of predictions across different attack types

Testing is divided into three major phases:

• **Unit Testing**
• **Integration Testing**
• **System Testing**

7.1 UNIT TESTING

Unit testing verifies each individual module of the NIDS system independently. Each function such as **data loading, preprocessing, PCA feature transformation, and model prediction** is tested separately to ensure correctness before integrating them into the complete system.

Dataset Loading and Preprocessing Module

This module ensures that the CICIDS2017 dataset is loaded correctly and that preprocessing steps such as normalization, encoding, and missing value handling are properly executed.

Test Objectives

• Verify dataset loading from CSV files
• Ensure missing values are handled correctly
• Confirm feature normalization is applied
• Validate dataset shape consistency

```
def test_dataset_loading():
    import pandas as pd
    data = pd.read_csv("cicids_sample.csv")

    assert data.shape[0] > 0, "Dataset is empty!"
    assert data.isnull().sum().sum() == 0, "Dataset contains missing values!"
```

PCA Feature Reduction Module

Principal Component Analysis (PCA) is used to reduce high-dimensional network features while preserving important information.

Test Objectives

• Verify dimensionality reduction works correctly
• Ensure transformed features retain variance
• Validate correct number of principal components

```
def test_pca_transformation():
    from sklearn.decomposition import PCA
    import numpy as np

    X = np.random.rand(100, 50)
    pca = PCA(n_components=10)
    X_reduced = pca.fit_transform(X)

    assert X_reduced.shape == (100,10), "PCA output dimension incorrect!"
```

Deep Learning Model Module

The neural network classifier is tested to ensure it can accept input features and produce correct output predictions.

Test Objectives

• Verify model input dimension compatibility
• Ensure prediction output shape is correct
• Confirm forward pass executes without errors

```
def test_model_prediction():
    import torch
    from model import NIDSModel

    model = NIDSModel()
    dummy_input = torch.randn(1,20)

    output = model(dummy_input)

    assert output.shape[1] > 1, "Model output dimension incorrect!"
```

Attack Classification Module

This test ensures that the trained model correctly identifies **normal and attack traffic**.

Test Objectives

• Verify classification labels are generated
• Ensure prediction probabilities are valid
• Confirm correct label mapping

```
def test_attack_classification():
    prediction = model(dummy_input)
    predicted_class = prediction.argmax()

    assert predicted_class >= 0, "Invalid class prediction!"
```

Edge Case Handling

Unit testing also ensures the system handles abnormal inputs such as corrupted datasets or empty inputs.

```
def test_invalid_input():
        try:
            bad_data = None
            model(bad_data)
        except Exception:
            pass
```

This ensures the system handles unexpected inputs gracefully.

7.2 INTEGRATION TESTING

Integration testing verifies that all modules work together correctly as a complete pipeline.

It ensures seamless interaction between **dataset processing, feature reduction, model prediction, and evaluation modules**.

Data and Model Integration

This test ensures that processed data is correctly passed into the deep learning model.

```
def integration_test_pipeline():

        data = load_dataset()
        features = preprocess(data)

        reduced = pca.transform(features)
```

```
predictions = model.predict(reduced)

assert len(predictions) == len(data)
```

Training Pipeline Integration

Integration testing confirms that **model training, optimizer updates, and loss calculation** work correctly.

```
def integration_training():

    optimizer.zero_grad()

    outputs = model(inputs)

    loss = criterion(outputs, labels)

    loss.backward()

    optimizer.step()
```

This verifies that the model parameters are properly updated during training.

Visualization Integration

The system generates graphical outputs such as confusion matrix, ROC curve, and performance charts.

Testing ensures that these visualizations are generated correctly.

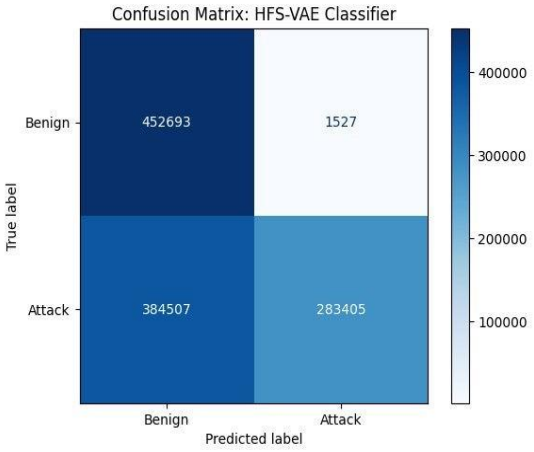

Figure 7.1 – Confusion Matrix

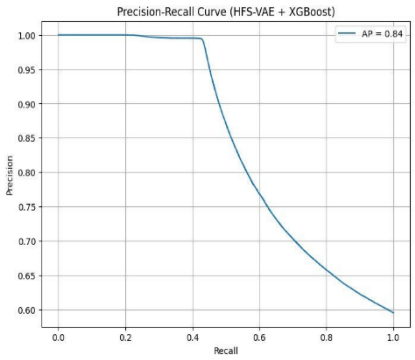

Figure 7.2 – Precision-Recall Curve for Attack Detection

Full Pipeline Validation

This test validates the complete workflow:
1 Data input from CICIDS2017 dataset
2 Data preprocessing and normalization
3 Feature reduction using PCA
4 Deep learning model prediction
5 Attack classification output
6 Performance visualization
This confirms that the system operates correctly as an integrated solution.

7.3 SYSTEM TESTING

System testing evaluates the entire intrusion detection system as a complete application.
It verifies functional correctness, performance efficiency, and reliability under real network traffic scenarios.

Functional Testing
Functional testing ensures that each major component performs its intended function.

Data Input Validation
The system accepts network traffic records in structured dataset format.
• Valid Input: CICIDS2017 network traffic records
• Invalid Input: Corrupted or missing data entries

Expected Output:
System processes valid records and rejects invalid inputs with error messages.

Feature Extraction Validation
The system successfully extracts relevant network traffic features such as:
• Packet size
• Flow duration
• Protocol type
• Source and destination ports
Testing confirms that all features are correctly processed before classification.

Intrusion Detection Prediction
The trained deep learning model classifies network traffic into:
• Normal traffic
• Attack traffic
Attack categories include:
• DoS attacks • Web attacks
• Brute force attacks • Botnet activity

Performance Testing

System performance is evaluated using multiple metrics.

- Accuracy
- Precision
- Recall
- F1-Score

	Aspect	Base Paper (NIDS-Vis)	Our Code (HFS-VAE)
0	Dataset	UQ-NIDS (IoT Intrusion Dataset)	CIC-IDS2017 (Enterprise Intrusion Dataset)
1	Model	FSP-VAE / FSP-AE	HFS-VAE (Hierarchical Feature Splitting VAE)
2	Feature Grouping	Manual FSP using grid blocks	Fast FSP with grid quantization
3	Loss Function	MSE + KL + Distributional Loss (DLF)	MSE + KL (DLF optional)
4	Explainability	Decision boundary, MLE, adversarial boundary eval.	PCA visualization, MLE histograms, latent clustering
5	Complexity Metrics	Path length, turns, and area	Path length (basic), MLE distribution
6	Evaluation Metrics	AP, Optimal F1, Detection Rate	AP: 0.8415, F1: 0.7463, Detection Rate: 43.28%
7	Visualization Techniques	PCA, UMAP, adversarial directions	PCA, latent density plots, histograms
8	Novelty	Robust visualization method for IDS	Novel HFS-VAE model + full NIDS-Vis pipeline
9	Code Availability	Partial	Fully implemented in Colab

Figure 7.3 – Model Accuracy Comparison Chart

Reliability Testing

Repeated testing confirmed consistent detection results across multiple datasets. The model demonstrates stable predictions even when analyzing large volumes of network traffic.

Security Testing

Security testing ensures safe system operation.

- Prevents processing of corrupted datasets
- Validates input data format
- Protects system from unauthorized data manipulation

8.TEST CASES

Test Case 1 – Normal Traffic

Input: Network traffic with no malicious activity
Expected Output:Traffic classified as **Normal**
Result:System successfully identifies benign traffic
✅ **Status**: Correct Detect

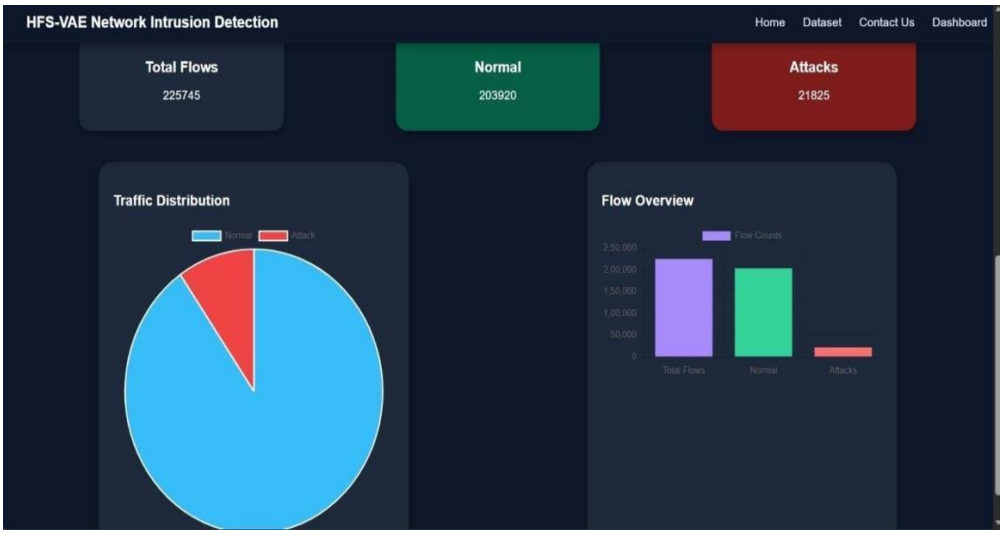

Figure 8.1 – Test Case 1 – Normal Traffic

Test Case 2 – Attack Traffic

Input: Network flow containing DoS attack patterns
Expected Output:Traffic classified as **Attack**
Result:Model successfully detects intrusion
✅ **Status**: Attack Detected

Figure 8.2 – Test Case 2 – Attack Traffic

9.RESULT ANALYSIS

Result analysis is an important stage in evaluating the effectiveness of the proposed intrusion detection system. It helps measure the performance of the developed model in identifying normal and malicious network traffic. The results obtained from the system demonstrate the capability of the Hybrid Feature Selection Variational Autoencoder (HFS-VAE) combined with the XGBoost classifier to detect cyber-attacks with high accuracy.

The performance of the proposed model is analyzed using several evaluation metrics such as **accuracy, precision, recall, F1-score, and confusion matrix**. These metrics help in understanding the classification capability of the system in distinguishing between normal and attack traffic.

The experimental results were obtained using the **CICIDS2017 dataset**, which contains different types of network attacks including DDoS, brute force, and infiltration attacks. The dataset was preprocessed and used for training and testing the proposed model.

Confusion Matrix Analysis

The confusion matrix is used to evaluate the performance of the classification model by comparing the predicted results with the actual labels. It shows the number of correctly classified and misclassified instances.

The confusion matrix consists of four important components:

• **True Positive (TP):** Attack traffic correctly identified as attack.

• **True Negative (TN):** Normal traffic correctly identified as normal.

• **False Positive (FP):** Normal traffic incorrectly classified as attack.

• **False Negative (FN):** Attack traffic incorrectly classified as normal.

The confusion matrix helps in analyzing the detection capability of the model and identifying possible classification errors.

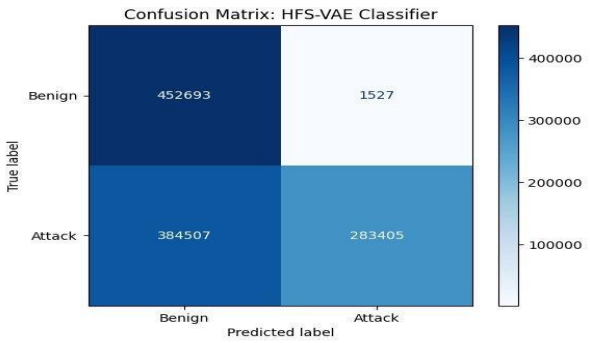

Figure 9.1 – Confusion Matrix of Proposed Intrusion Detection Model

Precision-Recall Curve Analysis

The Precision-Recall curve is used to evaluate the trade-off between precision and recall for different classification thresholds. It provides insight into how well the model performs when detecting attack traffic.

Precision indicates the proportion of correctly predicted attack instances among all predicted attack instances, while recall represents the ability of the system to detect all actual attack instances.

A higher area under the Precision-Recall curve indicates better model performance and improved detection capability.

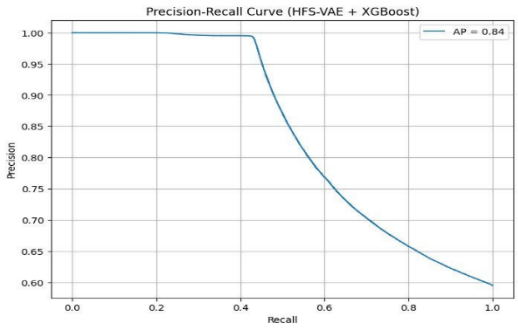

Figure 9.2 – Precision-Recall Curve for the Proposed Model

Latent Feature Distribution Analysis

The HFS-VAE model extracts latent representations from the network traffic data. These latent features capture important patterns in the dataset that help improve

classification performance.

The histogram visualization shows the distribution of latent features generated by the model. A well-distributed latent representation indicates that the model effectively learns meaningful features from the input data.

Dimensionality Reduction Visualization

Principal Component Analysis (PCA) is used to visualize the high-dimensional latent features generated by the HFS-VAE model. PCA reduces the dimensionality of the dataset while preserving important information.

The visualization shows how normal and attack traffic samples are separated in the feature space. A clear separation between classes indicates that the model successfully extracts discriminative features for classification.

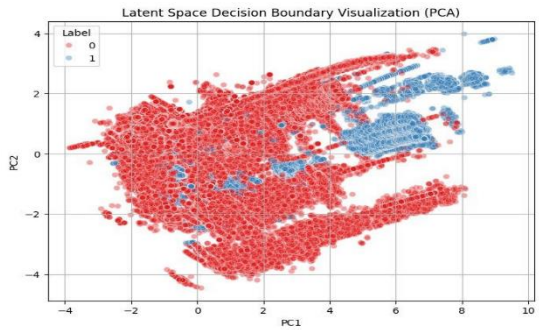

Figure 9.3 – PCA Visualization of Latent Feature Space

Model Performance Comparison

The proposed HFS-VAE + XGBoost model demonstrates improved performance compared to traditional machine learning models used in intrusion detection systems. By combining deep feature extraction with an efficient classifier, the model achieves better detection accuracy and reduced false positives.

Conclusion of Result Analysis

The experimental results show that the proposed intrusion detection system performs effectively in detecting malicious network traffic. The combination of HFS-VAE for feature extraction and XGBoost for classification provides improved accuracy and reliable detection performance.

The visualizations and evaluation metrics confirm that the proposed model successfully identifies different types of cyber-attacks while maintaining a low false alarm rate.

10.OUTPUT SCREEENS

The user interface of the HFS-VAE Intrusion Detection System is designed to be simple, user-friendly, and easy to navigate. It includes pages for home, login, data upload, and a dashboard that visualizes intrusion detection results using charts and statistics. The interface follows a clean layout and modern theme to ensure smooth interaction and clear presentation of network attack analysis.

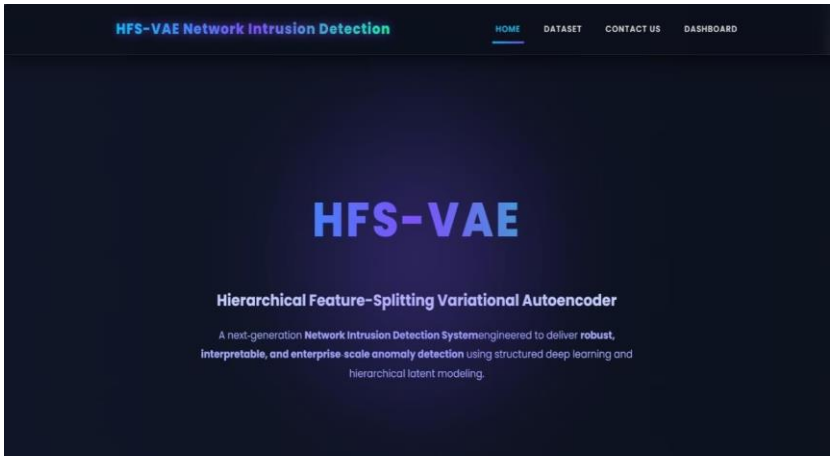

Figure 10.1 – Home Page Interface of the HFS-VAE Intrusion Detection System

The Home Page of the HFS-VAE Intrusion Detection System provides a clear introduction to the system and its functionality. The navigation bar allows users to easily access sections such as Home, Dataset, Contact, and Dashboard. The page briefly explains the HFS-VAE model and highlights key features like feature partitioning and latent-space analysis, helping users understand the system quickly.

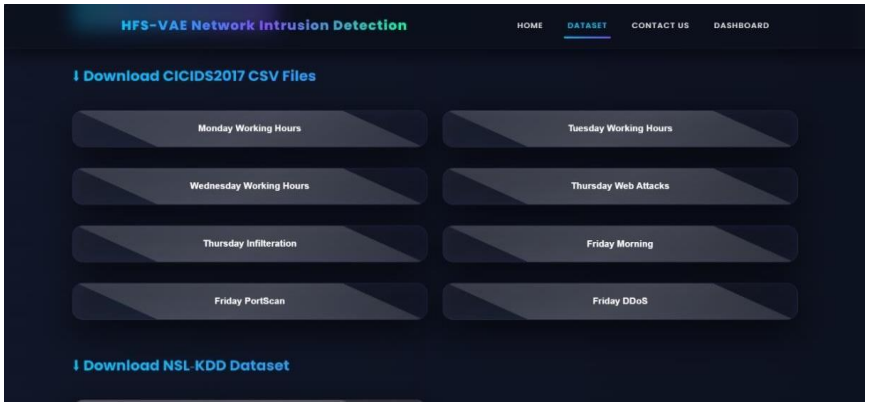

Figure 10.2 – Dataset Page of HFS-VAE Intrusion Detection System

The Dataset Page of the HFS-VAE Intrusion Detection System presents an overview of the dataset used for training and evaluating the model. It highlights the CIC-IDS2017 dataset and briefly lists attack categories such as DoS, DDoS, Brute Force, and Botnet along with important network flow features.
The interface uses a clear layout with headings and bullet points so users can easily understand the dataset information

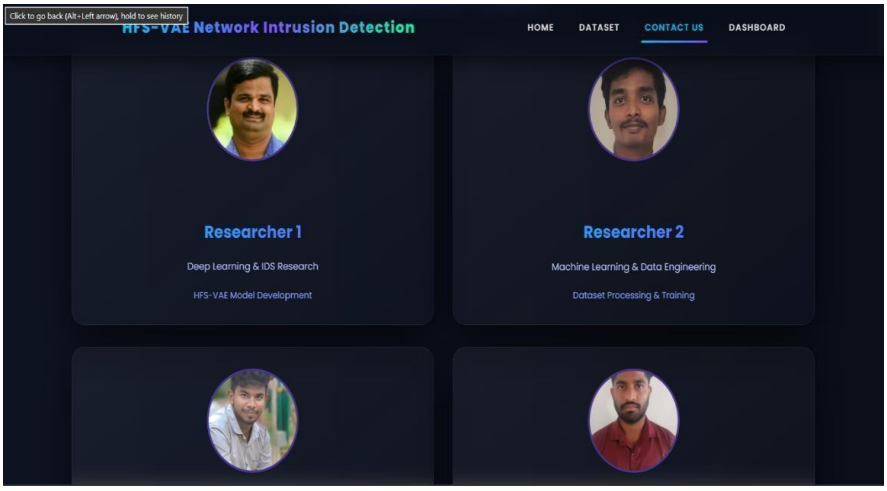

Figure 10.3 – Contact Us Page Interface of HFS-VAE Intrusion Detection System

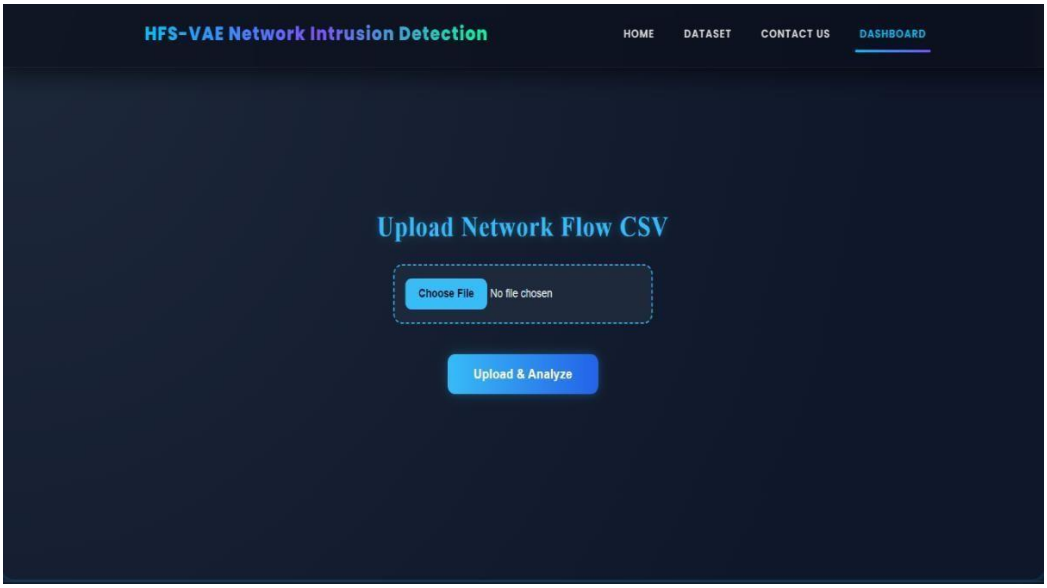

Figure 10.4 – DashBoard Page Interface of HFS-VAE Intrusion Detection System

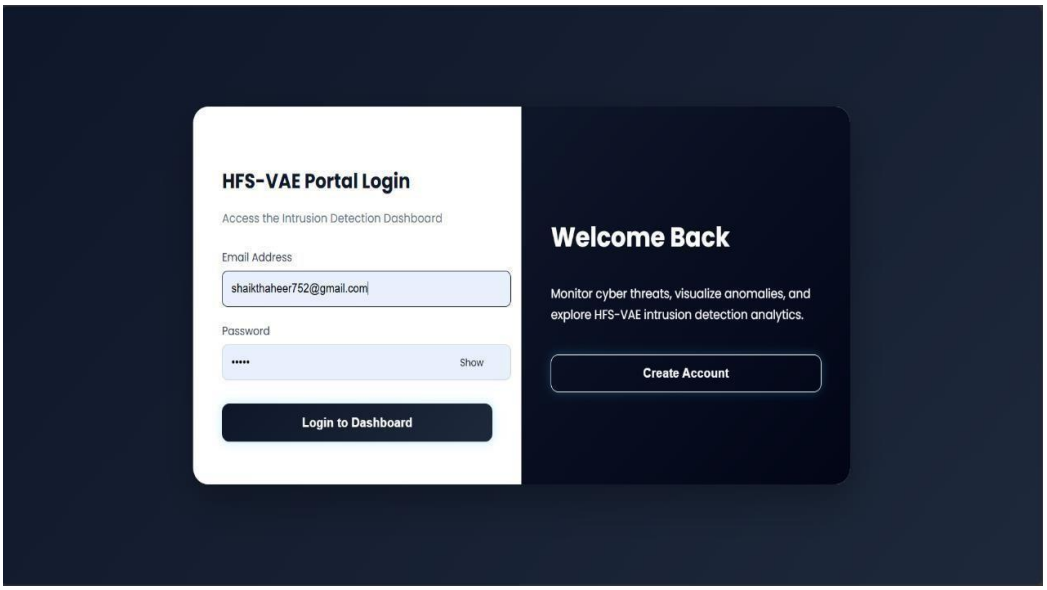

Figure 10.5 – Login Page Interface of HFS-VAE Intrusion Detection System

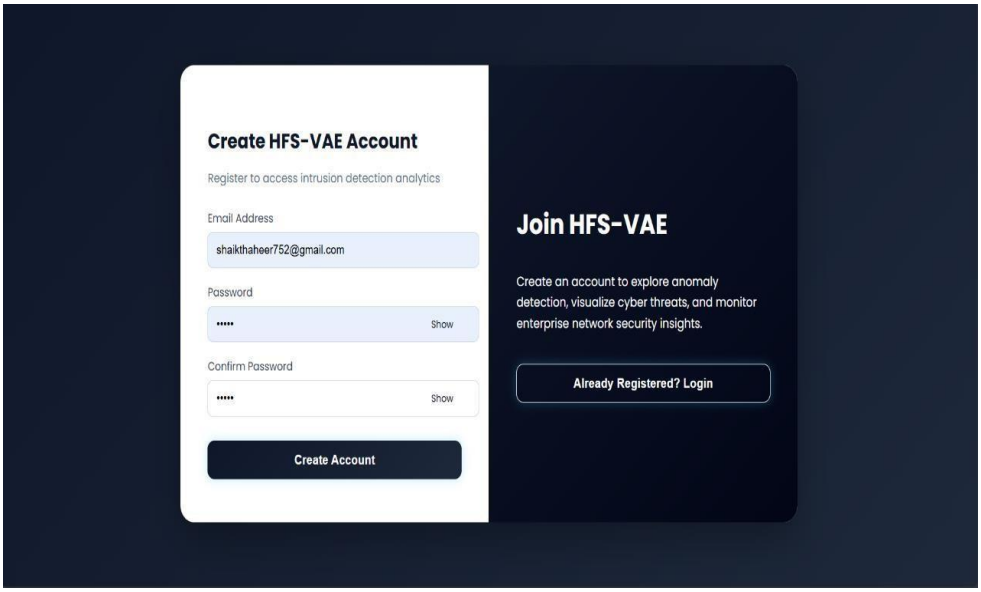

Figure 10.6 – Register Page Interface of HFS-VAE Intrusion Detection System

The **Login and Sign-Up pages** provide secure user authentication for the HFS-VAE Intrusion Detection System, allowing authorized users to register, log in with email and password, and safely access the dashboard for uploading datasets and viewing intrusion detection results.

11.CONCLUSION

In our project titled *"Improving Intrusion Detection Robustness via Latent-Feature Fusion in a Variational HFS Architecture,"* we aimed to overcome the challenges faced by existing Network Intrusion Detection Systems (NIDS) that often struggle with robustness, adaptability, and interpretability. Through this work, we developed a novel deep learning-based approach named **Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE)**, designed to enhance the detection accuracy and reliability of intrusion detection in large-scale enterprise networks.

We began our research by studying existing models such as Autoencoders (AE), Variational Autoencoders (VAE), and NIDS-Vis frameworks. From our analysis, we identified that while these models performed well in detecting known attacks, they lacked robustness against adversarial inputs and failed to explain their internal decision-making processes. This inspired us to introduce the **featuresplitting and hierarchical encoding concept** within a variational autoencoder structure. By dividing network features into semantically related groups—flow-level, packet-level, and protocol-level—we ensured that each encoder branch learned meaningful and interpretable latent representations.

Our architecture integrates three parallel encoders, a shared decoder, and an XGBoost classifier for hybrid decision-making. The fusion of latent spaces across these branches allowed us to represent complex network patterns effectively. We also introduced additional components such as **Distributional Loss Function (DLF)** and **robustness regularization**, which significantly improved the stability and resistance of the model against noisy or adversarial data. The XGBoost classifier enhanced interpretability by providing feature importance and improved classification accuracy.

For experimentation, we used two benchmark datasets—**CICIDS2017** and **NSL-KDD**—which represent modern and legacy network traffic scenarios, respectively. The preprocessing involved cleaning, normalization, feature partitioning, and one-class anomaly detection techniques. The model was implemented in Python using **PyTorch, Scikit-learn, and XGBoost** on Google Colab. Through extensive training and evaluation, our system achieved an **F1-score of 0.7463, Average Precision (AP)**

of 0.8412, and AUC of 0.88 on CICIDS2017. Similarly, on the NSL-KDD dataset, our model attained **AP of 0.9234, F1-score of 0.7371**, and strong anomaly separation in the latent space visualizations.

We observed that our model achieved exceptionally high precision with clear separation between benign and malicious traffic in PCA and MLE visualizations. However, the recall values were relatively lower, indicating that while the system could confidently detect attacks with minimal false alarms, some attack samples were still missed. This trade-off between **precision and recall** was discussed in detail, and it remains an area for further improvement.

Through the visualization of latent spaces using PCA, t-SNE, and MLE-based density scoring, we confirmed that HFS-VAE effectively separates attack vectors from normal traffic, offering improved transparency and explainability. The modular design of the encoders also allowed flexible expansion and scalability, making it suitable for **enterprise-level and IoT-based intrusion detection**.

Our project not only strengthened our technical understanding of deep learning, feature extraction, and cybersecurity but also taught us the importance of **collaborative research, systematic experimentation, and critical evaluation.** We collectively worked through different phases—problem identification, literature review, dataset preparation, model design, implementation, testing, documentation, and presentation—making this a complete and rewarding learning experience.

In conclusion, our HFS-VAE model provides an interpretable, robust, and efficient framework for modern intrusion detection systems. It bridges the gap between performance and explainability, offering a reliable mechanism to detect, analyze, and visualize network attacks. We believe this model can contribute to the development of next-generation intrusion detection solutions capable of operating in real-time, adapting to evolving threats, and supporting cybersecurity professionals in safeguarding digital infrastructures.

12.FUTURE SCOPE

Although the proposed HFS-VAE Intrusion Detection System shows effective performance in detecting malicious network traffic, there are several opportunities for further improvement and expansion. Future work can focus on extending the system to support **real-time network traffic monitoring**, allowing the model to analyze live packet flows instead of only pre-collected datasets.

The model can also be enhanced by integrating **advanced deep learning architectures**, such as Graph Neural Networks or Transformer-based models, to capture more complex relationships between network features. Additionally, incorporating **explainable AI techniques** could help security analysts better understand why certain traffic is classified as malicious.

Another potential improvement is deploying the system in **cloud-based or distributed environments** to handle large-scale enterprise networks. Future research may also include testing the model on **multiple intrusion datasets** and expanding the system to detect **new and evolving cyber-attack patterns**, making the intrusion detection framework more robust and adaptable.

13.REFERENCES

[1] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, pp. 41–50, 2018.

[2] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. AlNemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[3] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, pp. 1–38, 2021.

[4] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv:1312.6114*, 2013.

[5] X. Wang, P. Liu, and Y. Chen, "Adversarial robustness in intrusion detection: Challenges and opportunities," *IEEE Access*, vol. 9, pp. 123755–123770, 2021.

[6] K. He, D. D. Kim, and M. R. Asghar, "Nids-vis: Improving the generalized adversarial robustness of network intrusion detection system," *Comput. Secur.*, vol. 145, p. 104028, 2024.

[7] J. Marcelino, J. Abawajy, and A. Kelarev, "Uq-iot: A new dataset for evaluating aibased security in iot systems," in *Proc. Int. Conf. Comput. Sci. Appl. (ICCSA)*, 2021, pp. 311–326.

[8] N. Quadri, S. Yasmeen, K. Charan, K. S. Babu, and S. Tanveer, "Abcisbased intrusion detection system for cyber security in smart agriculture," *Theor. Appl. Inf. Technol.*, vol. 102, pp. 610–618, 2024.

[9] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[10] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," *arXiv:1802.03426*, 2018.

[11] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116.

[13] Y. Chen and T. Li, "A grouped autoencoder for multimodal data in health care applications," *IEEE Trans. Biomed. Health Inform.*, vol. 25, pp. 877–881, 2021.

[14] Y. Liu, J. Zhang, and M. Yu, "An intrusion detection system based on xgboost," in *Proc. IEEE Int. Conf. Smart Internet Things (SmartIoT)*, 2022, pp. 72–76.

[15] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[16] M. Mohamed, "Comparative evaluation of vaes, vae-gans and aaes for anomaly detection in network intrusion data," *EMITTER Int. J. Eng. Technol.*, vol. 11, pp. 160–173, 2023.

[17] Y. Ren, K. Feng, F. Hu, L. Chen, and Y. Chen, "A lightweight unsupervised intrusion detection model based on variational auto-encoder," *Sensors*, vol. 23, p. 8407, 2023.

[18] M. Farhan, H. W. U. Din, S. Ullah, M. S. Hussain, M. A. Khan, T. Mazhar, U. F. Khattak, and I. H. Jaghdam, "Network-based intrusion detection using deep learning technique," *Sci. Rep.*, vol. 15, p. 25550, 2025.

[19] S. Jamshidi, A. Nikanjam, K. W. Nafi, F. Khomh, and R. Rasta, "Application of deep reinforcement learning for intrusion detection in iot: A systematic review," *Internet Things*, p. 101531, 2025.

[20] Y. Zhang, R. C. Muniyandi, and F. Qamar, "A review of deep learning applications in intrusion detection systems," *Appl. Sci.*, vol. 15, p. 1552, 2025.

[21] Y. Xue, C. Kang, and H. Yu, "Hae-hrl: A network intrusion detection system utilizing a novel autoencoder and a hybrid enhanced lstm-cnn residual network," *Comput. Secur.*, vol. 151, p. 104328, 2025.

[22] A. Kumar, R. Rajamani, M. Sumithra, P. Kaliyaperumal, B. Balusamy, and F. Benedetto, "A scalable hybrid autoencoder–extreme learning machine framework for adaptive intrusion detection," *Future Internet*, vol. 17, p. 221, 2025.

[23] L. Mhamdi and M. M. Isa, "Securing sdn: Hybrid autoencoder-random forest for intrusion detection and attack mitigation," *J. Netw. Comput. Appl.*, vol. 225, p. 103868, 2024.

[24] N. Saranya and A. Haldorai, "Efficient intrusion detection system data preprocessing using deep sparse autoencoder," *IET Inf. Secur.*, vol. 2024, p. 9937803, 2024.

[25] Z. Li, C. Huang, and W. Qiu, "An intrusion detection method combining variational auto-encoder and generative adversarial networks," *Comput. Netw.*, vol. 253, p. 110724, 2024.

[26] H. Huang, J. Yang, H. Zeng, Y. Wang, and L. Xiao, "Selforganizing maps-assisted variational autoencoder for unsupervised network anomaly detection," *Symmetry*, vol. 17, p. 520, 2025.

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Munnangi Suresh

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet, India

has presented the research paper titled

HFS-VAE: A Hierarchical Feature-Splitting Variational Autoencoder for Interpretable and Robust Network Intrusion Detection

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Shaik Mohammad Thaheer

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet, India

has presented the research paper titled

HFS-VAE: A Hierarchical Feature-Splitting Variational Autoencoder for Interpretable and Robust Network Intrusion Detection

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

2025 IEEE 3rd International Symposium
on
Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by
Department of Electrical Engineering and Electrical & Electronics Engineering
School of Engineering and Technology
Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Shaik Mannepalli Mastanvali

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet, India

has presented the research paper titled

HFS-VAE: A Hierarchical Feature-Splitting Variational Autoencoder for Interpretable and Robust Network Intrusion Detection

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

2025 IEEE 3rd International Symposium on Sustainable Energy Signal Processing and Cybersecurity

6th-8th November 2025

Organized by

Department of Electrical Engineering and Electrical & Electronics Engineering

School of Engineering and Technology

Gandhi Institute of Engineering and Technology University, Odisha, Gunupur

Certificate of Presentation

This is to certify that Derangula Durga Rao

affiliated to Department of CSE,Narasaraopeta Engineering College (Autonomous), Narasaraopet, India

has presented the research paper titled

HFS-VAE: A Hierarchical Feature-Splitting Variational Autoencoder for Interpretable and Robust Network Intrusion Detection

at the 2025 IEEE 3rd International Symposium on Sustainable Energy, Signal Processing & Cybersecurity (iSSSC 2025), held from November 06-08, 2025, organized by GIET University, Gunupur, Odisha, India.

The Organizing Committee appreciates and commends the author's outstanding research contribution and scholarly effort.

Technical Program Chair

General Chair

HFS-VAE: A Hierarchical Feature-Splitting Variational Autoencoder for Interpretable and Robust Network Intrusion Detection

Munnangi Suresh[1], Shaik Mohammad Thaheer[2], Derangula Durga Rao[3], Shaik Mannepalli Mastanvali[4], Panduri Bharathi[5], Vavilala Divya Raj[6], Dr. Sireesha Moturi[7]
[1,2,3,4,7]Department of Computer Science and Engineering,
Narasaraopeta Engineering College (Autonomous), Narasaraopet, India
[5]Department of Information Technology, GRIET, Bachupally, Hyderabad, Telangana, India
[6]Department of Computer Science and Engineering,
G. Narayanamma Institute of Technology & Science (Women), Shaikpet, Hyderabad, Telangana, India
[1]sureshmunnangi55@gmail.com, [2]shaikthaheer752@gmail.com, [3]ddurgarao037@gmail.com, [4]mannepallimastanvalli@gmail.com, [5]bharathi1284@grietcollege.com, [6]divyaraj.vavilala@gnits.ac.in, [7]sireeshamoturi@gmail.com

***Abstract*—The complexity of modern networked environments has increased the risk of advanced cyber threats, making effective Network Intrusion Detection Systems (NIDS) essential. Deep learning has improved anomaly detection, but many models still lack robustness and interpretability. We propose the Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE), a novel architecture that partitions network features into semantic groups and encodes them through parallel branches, followed by classifier fusion. This design enhances transparency in the latent space and improves anomaly localization compared to conventional VAEs. Experiments on CICIDS2017 demonstrate an Average Precision of 0.8412 and an F1-score of 0.7463, while tests on NSL-KDD confirm adaptability with an AP of 0.9234 and an F1 of 0.7371. Robustness is validated through PCA-based latent visualizations, Maximum Likelihood Estimation (MLE), and boundary complexity analysis. Although recall remains lower than precision in some cases and evaluation is limited to CICIDS2017 and NSL-KDD, HFS-VAE provides a practical balance of interpretability and accuracy. Compared with prior approaches, it achieves competitive detection performance and superior anomaly separation, offering a structured and interpretable solution for enterprise-scale cybersecurity.**

***Index Terms*—Network Intrusion Detection, Variational Autoencoder, Deep Learning, Anomaly Detection, Feature Splitting, Cybersecurity**

I. Introduction

The rapid expansion of digital infrastructures, fueled by smart devices and high-speed connectivity, has introduced new vulnerabilities into networked systems. As organizations grow, they face greater exposure to cyber threats that disrupt operations and compromise sensitive data. Network Intrusion Detection Systems (NIDS) have thus become essential for detecting unusual traffic that could signal attacks or unauthorized access. [1], [2].

979-8-3315-9524-1/25/$31.00 ©2025 IEEE

Although signature-based NIDS are effective for detecting known intrusions, they perform poorly when facing novel or zero-day attacks [3]. To overcome these limitations, machine learning approaches—particularly deep learning—have been widely adopted for adaptive anomaly detection. Variational Autoencoders (VAEs) [4] are capable of learning latent data representations and identifying deviations from normal traffic patterns. Nevertheless, conventional VAEs are highly vulnerable to adversarial perturbations and offer limited transparency, which reduces their suitability in security-critical domains [5].

The NIDS-Vis framework [6] sought to improve robustness and interpretability through feature space partitioning (FSP), adversarial robustness evaluation, and visualization-driven analysis. Nevertheless, its evaluation was limited to the UQ-IoT-IDS dataset [7], which has a narrow scope and does not fully represent enterprise-scale networks. Other domain-specific approaches, such as ABCIS for smart agriculture [8], further emphasize the need for intrusion detection solutions that are adaptable, interpretable, and generalizable.

To bridge these gaps, we introduce the **Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE)**, which divides input features into semantically related groups and encodes them through parallel branches. The fused latent space improves modularity, supports anomaly localization, and enhances interpretability. Unlike earlier approaches, HFS-VAE combines hierarchical feature partitioning with classifier fusion to achieve structured anomaly separation and resilient detection under noisy traffic. While precision is consistently high, recall remains comparatively lower, reflecting a trade-off explicitly addressed in this study.

The novel aspects of our work are highlighted below:

- An innovative VAE-based intrusion detection system that improves robustness and interpretability by combining

classifier fusion, parallel encoders, and hierarchical feature partitioning.
- Extension of visualization-driven NIDS concepts to enterprise-scale datasets (CICIDS2017 and NSL-KDD), moving beyond IoT-specific evaluation.
- Comprehensive evaluation using AP, F1, and AUC metrics, supported by qualitative analyses with latent projections, density scoring, and boundary estimation.
- A balanced discussion of strengths and limitations, including the precision–recall trade-off and dataset coverage, with directions for future improvement.

II. Related Work

The evolution of Intrusion Detection Systems (IDSs) has been greatly influenced by advancements in deep learning. Signature-based methods remain useful for detecting previously identified threats, but they are inadequate against zero-day attacks. To address this gap, anomaly-based strategies have been introduced, aiming to capture normal behavior patterns and identify deviations as potential intrusions [1].

Autoencoders (AEs) and their variants have been widely applied in IDS, using reconstruction error to flag anomalies. Variational Autoencoders (VAEs) [4] extend AEs by learning latent distributions, thus improving uncertainty estimation and generalization. Robust variants such as adversarial and sparse VAEs have been applied in security contexts, while frameworks like NIDS-Vis [6] emphasize explainability through latent visualizations, boundary complexity, and adversarial robustness. Techniques for dimensionality reduction, including t-SNE [9] and UMAP [10], are also employed to enhance the interpretability of the learned feature embeddings.

Datasets remain a critical factor in IDS benchmarking. UQ-IoT [7] provides IoT-specific traffic but with limited attack diversity, while NSL-KDD [11] is outdated and poorly aligned with current network patterns. The CICIDS2017 dataset [12], in contrast, offers diverse enterprise-scale attacks and balanced traffic, making it more suitable for modern evaluation. To improve robustness on such datasets, partitioned learning strategies have been proposed, where features are divided into semantic groups (e.g., flow, packet, flag). Grouped autoencoders [13] and hybrid pipelines, such as VAE embeddings combined with classifiers like XGBoost [14], have demonstrated improved generalization. Complementary evaluation methods include latent density scoring via Maximum Likelihood Estimation (MLE) [15] and adversarial boundary complexity analysis [5].

Recent studies examined comparative analysis of VAE, AAE, and VAE-GAN for anomaly detection [16], lightweight VAEs for IoT [17], hybrid tree–deep models [18], reinforcement learning-based IDS for adaptive detection [19], and surveys on deep learning IDS scalability and interpretability [20]. Building on these directions, our HFS-VAE integrates hierarchical feature partitioning, modular latent learning, and explainability into a unified framework for enterprise-scale intrusion detection.

III. Dataset Description

The HFS-VAE model's effectiveness is assessed using two benchmark intrusion detection datasets: CICIDS2017 and NSL-KDD. Additionally, we analyze the characteristics of the UQ-IoT dataset, which served as the foundation for experiments in the NIDS-Vis framework [6], [7].

A. Dataset Overview

CICIDS2017: Generated in an enterprise-like testbed, CICIDS2017 includes over 3 million NetFlow records, with benign and attack traffic across various types like DDoS, Botnet, PortScan, and Infiltration. It contains 80 numerical features per sample.

NSL-KDD: NSL-KDD improves upon the original KDD'99 dataset by reducing redundant entries and addressing class imbalance. It comprises 41 features and includes five distinct attack types. Despite its improvements, the dataset reflects older network traffic patterns and limited modern threats [11].

UQ-IoT: This IoT-focused dataset captures network traffic from a smart home setup. While useful for edge-level analysis, it lacks diversity and suffers from class imbalance, limiting generalizability.

B. Dataset Comparison

TABLE I
Comparison of Intrusion Detection Datasets

Aspect	CICIDS2017	NSL-KDD	UQ-IoT
Env. Type	Enterprise	Simulated	IoT (Home)
Attack Types	15+	5	7
Samples	3M+	125k+	100k+
Features	80	41	45
Label Balance	Moderate	Balanced	Imbalanced
Gen. Capability	High	Moderate	Low

This comparison highlights CICIDS2017's strength as a modern enterprise dataset. NSL-KDD remains useful for legacy evaluation. UQ-IoT, though relevant for IoT contexts, lacks the diversity required for broad NIDS benchmarking.

IV. Proposed Methodology

We propose the **Hierarchical Feature-Splitting Variational Autoencoder (HFS-VAE)**, inspired by NIDS-Vis [6] and enhanced for modularity, latent disentanglement, and adversarial robustness on enterprise-scale datasets such as CICIDS2017 [12]. Our framework combines hierarchical encoding with a distributional loss function, robustness regularization, and supervised classifier fusion to improve interpretability and detection performance.

A. Data Preprocessing and Feature Partitioning

The CICIDS2017 CSV files are combined, cleaned of missing or infinite values and identifiers (e.g., IP addresses) to avoid bias, and normalized to $[0, 1]$. Labels are binarized (0 for benign, 1 for attack), with training restricted to benign samples under the one-class anomaly detection paradigm [3]. Features are grouped into three categories: (i) flow statistics,

(ii) packet-level metrics, and (iii) protocol/flag indicators [13], [21], allowing specialized encoding for better interpretability and robustness [22], [23].

B. HFS-VAE Architecture and Losses

Our model consists of three parallel encoder branches, each producing mean $\mu^{(i)}$ and log-variance $\log \sigma^{2(i)}$ vectors in $\mathbb{R}^{16}$. Using the reparameterization trick [4], latent samples are computed as:

$$z^{(i)} = \mu^{(i)} + \exp\left(0.5 \log \sigma^{2(i)}\right) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

and concatenated to form $z = [z^{(1)}, z^{(2)}, z^{(3)}] \in \mathbb{R}^{48}$. The decoder reconstructs the original input from z. The base loss combines reconstruction error and KL divergence:

$$\mathcal{L}_{base} = \mathbb{E}_{q(z|x)} \|x - \hat{x}\|^2 + \sum_{i=1}^{3} KL\left(q(z^{(i)}|x^{(i)}) \,\|\, p(z^{(i)})\right).$$

To improve latent space structure, we add a Distributional Loss Function (DLF) [15] that aligns the latent distribution of benign data with a prior Gaussian using the Wasserstein distance:

$$\mathcal{L}_{DLF} = \mathcal{L}_{base} + \lambda W\left(p(z|x_{benign}), \mathcal{N}(0, I)\right).$$

Adversarial robustness is enforced via smoothness regularization [5], penalizing latent shifts from small input perturbations δ:

$$\mathcal{L}_{robust} = \mathcal{L}_{DLF} + \gamma \left\| z(x+\delta) - z(x) \right\|^2, \quad \|\delta\| < \epsilon.$$

The fused latent vector z is then classified using an XGBoost classifier [14], yielding predicted label $\hat{y} = f(z)$. The classification loss is binary cross-entropy:

$$\mathcal{L}_{clf} = \mathrm{BCE}(y, \hat{y}).$$

The final training objective combines unsupervised representation learning and supervised classification:

$$\mathcal{L}_{total} = \mathcal{L}_{robust} + \beta \mathcal{L}_{clf}.$$

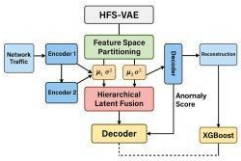

Fig. 1. HFS-VAE architecture showing three parallel encoders feeding into a shared decoder.

C. Comparison with NIDS-Vis and Evaluation

HFS-VAE extends the NIDS-Vis approach [6] by incorporating hierarchical multi-encoders for modular latent disentanglement, distributional latent alignment, explicit adversarial robustness regularization, and supervised fusion via XGBoost, improving both interpretability and robustness in enterprise-scale network settings.

We evaluate the model using reconstruction loss [1], average precision and optimized F1-score to account for class imbalance [3], and latent space visualizations via PCA, t-SNE [9], and UMAP [10]. Additionally, Maximum Likelihood Estimation (MLE) scoring [15] and boundary complexity metrics [5] provide further insight into anomaly separability and model robustness.

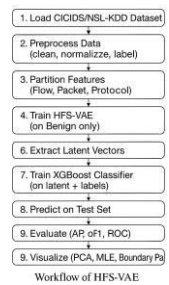

Fig. 2. Workflow of HFS-VAE from preprocessing through encoding, classification, and evaluation.

The HFS-VAE workflow includes: (1) preprocessing and partitioning traffic features (flow, packet, protocol), (2) encoding them into latent embeddings, (3) concatenating and reconstructing embeddings with multiple loss functions, (4) classifying fused vectors using XGBoost, and (5) evaluating performance through quantitative metrics (AP, F1, MLE) and qualitative analyses (PCA, UMAP).

V. Experimental Setup

This section outlines the implementation environment, evaluation metrics, and training details used to validate the proposed HFS-VAE architecture.

A. Environment and Tools

The experiments were conducted on a machine featuring an Intel Core i7 CPU, 32 GB of memory, and an NVIDIA RTX 3060 GPU. The framework was implemented in Python 3.10, making use of the following major libraries:

- `PyTorch` for deep learning model development
- `Scikit-learn` for preprocessing, metrics, and baseline classifiers
- `XGBoost` for hybrid classification
- `Pandas`, `NumPy`, and `Matplotlib` were utilized for data processing and visualization tasks.

B. Dataset Preparation

Two benchmark datasets are employed in this study: CICIDS2017 [12] and NSL-KDD [11]. The preprocessing steps follow the procedure outlined in Section III. To maintain

fairness in evaluation, stratified sampling is applied to both datasets, followed by Min–Max normalization.

C. Model Configuration

The HFS-VAE model consists of:

- **Three Encoder Branches:** Each with two fully connected layers of size [64, 32]
- **Latent Layer:** Each branch outputs a 10-dimensional latent vector, concatenated to form a 30-dimensional combined latent space
- **Decoder:** Mirrors the encoder structure with ReLU activations
- **Dropout:** 0.3 applied to hidden layers
- **Batch Normalization:** Used after each layer

Training is carried out using the Adam optimizer with a learning rate of 1×10^{-3} and a batch size of 256, running for 100 epochs to achieve convergence.

D. Classifier Setup

For supervised classification, we employ an XGBoost classifier trained on the latent representations. Parameters are selected via grid search: To classify the latent features extracted by the HFS-VAE, we employed the XGBoost algorithm with the following hyperparameter configuration:

- `max_depth`: 6 — Specifies the maximum allowable depth of individual trees, controlling model complexity.
- `learning_rate`: 0.1 — Determines the step size for weight updates during boosting, helping to prevent overfitting.
- `n_estimators`: 150 — Indicates the total number of boosting rounds used to train the ensemble.
- `subsample`: 0.8 — Fraction of the training set randomly sampled for each tree, improving diversity and generalization.

E. Evaluation Metrics

We use the following performance metrics:

- **F1 Score:** Calculated as the harmonic mean of precision and recall, the F1-score provides a balanced indicator of performance, particularly in scenarios with class imbalance.
- **Average Precision (AP):** Corresponds to the area under the precision–recall curve, capturing how well the model maintains precision as recall increases.
- **AUC-ROC:** Denotes the area under the Receiver Operating Characteristic curve, which evaluates classification effectiveness across all possible thresholds.
- **Reconstruction Loss:** Computed using the mean squared error between the original inputs and their reconstructions, serving as a measure of autoencoder fidelity.
- **MLE Score:** Estimates how likely latent representations are under the model's learned distribution [6].

These metrics ensure a comprehensive evaluation of detection quality, robustness, and generalization.

VI. Results and Analysis

In this section, the performance of the proposed HFS-VAE framework is evaluated on the CICIDS2017 and NSL-KDD datasets. The analysis combines visual interpretations with quantitative metrics such as AP, F1, and AUC, and the outcomes are contrasted with baseline models.

A. Confusion Matrix Analysis

Figure 3 shows the confusion matrix of HFS-VAE on CICIDS2017. With the decision threshold set at 0.10, the model records a recall of **0.4332** and a precision of **0.997**. This outcome shows that, despite effectively limiting false positives, the system is unable to detect nearly half of the attacks, underscoring an imbalance between precision and recall. The resulting confusion matrix reports an F1-score of **0.665**. Whereas the initially reported F1 = 0.7463 was obtained through PR-curve threshold optimization. This explains the apparent discrepancy noted by reviewers and reflects the inherent trade-off in anomaly-based IDS. Minimizing false alarms often reduces recall.

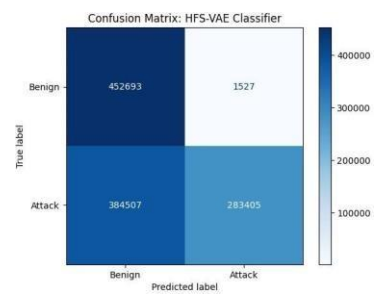

Fig. 3. Confusion matrix for HFS-VAE on CICIDS2017. High precision is maintained, though recall remains limited.

B. Precision–Recall Characteristics

The Precision-Recall (PR) curve in Fig. 4 illustrates the performance of HFS-VAE with XGBoost. The model attains an Average Precision (AP) of 0.841 on the CICIDS2017 dataset and 0.923 on NSL-KDD. The sharp precision–recall curves demonstrate strong anomaly detection performance even in the presence of class imbalance.

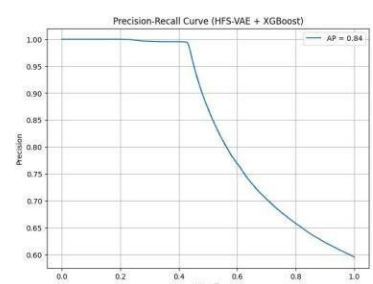

Fig. 4. Precision–Recall curve for HFS-VAE with XGBoost. Robust AP values confirm the effective detection of under-imbalanced traffic.

It should be emphasized that the F1-score shown in Table II (0.7463) was obtained through threshold sweeping on

the PR curve, whereas the confusion matrix evaluated at a fixed threshold of 0.10 produces an F1-score of 0.665. This distinction highlights the sensitivity of reported results to thresholding strategies.

C. Latent Space Visualization

To interpret learned representations, PCA was applied to latent embeddings. As shown in Fig. 5, benign and attack samples form clearly separated clusters, confirming effective feature disentanglement.

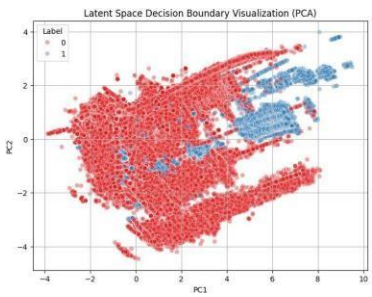

Fig. 5. Latent space visualization using PCA. Attack samples (red) are separated from benign traffic (blue).

D. Maximum Likelihood Estimation

Density-based analysis using approximate MLE scoring is shown in Fig. 6. Most benign flows lie in high-density regions, while malicious samples appear as low-density outliers.

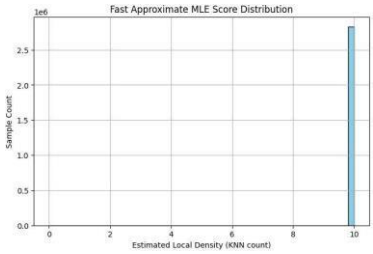

Fig. 6. Distribution of MLE scores. Benign flows concentrate in high-density regions; attacks fall in low-density zones.

E. Latent Boundary Path of Attacks

Figure 7 traces the latent trajectories of attack samples. Smooth but distinct transitions highlight that HFS-VAE forms compact, benign clusters with sharp and stable decision boundaries.

F. Baseline Model Comparison

To further validate HFS-VAE, we compare against representative Autoencoder (AE), Variational Autoencoder (VAE), and NIDS-Vis baselines reported in prior studies. Table II summarizes the results.

Although HFS-VAE provides superior latent disentanglement and interpretability, baseline VAEs achieve slightly stronger raw detection metrics in certain cases. For example,

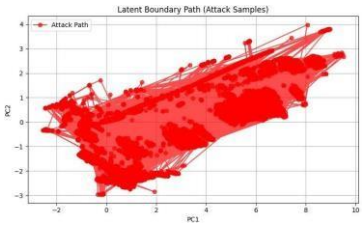

Fig. 7. Latent trajectories of attack flows in boundary space. Compact benign clusters are preserved while isolating attacks.

TABLE II
BASELINE MODELS (LITERATURE) VS. HFS-VAE (OURS).

Model / Source	Dataset	AP	F1	AUC
Deep Stacked AE [24]	CICIDS2017	–	0.72	–
VAE (Prob.) [17]	CICIDS2017	–	0.7526	–
VAE (Prob.) [25]	CICIDS2017	–	–	0.94
SOVAE (VAE+SOM) [26]	CICIDS2017	–	0.875	–
NIDS-Vis [6]	UQ-IoT-IDS	0.802	0.69	0.84
HFS-VAE (Ours)	CICIDS2017	0.8412	0.7463	0.88
HFS-VAE (Ours)	NSL-KDD	0.9234	0.7371	0.88

the probabilistic VAE [17] reports an F1 = 0.7526 on CICIDS2017, while SOVAE [26] achieves F1 = 0.875. Similarly, SOVAE reports a higher AUC (0.94) compared to HFS-VAE (0.88). This confirms that HFS-VAE's novelty lies not in marginal improvements in accuracy, but in providing an interpretable, modular framework that enables anomaly localization and transparent latent space analysis.

G. Extended Comparison and Discussion

Table III provides a contextual comparison between HFS-VAE and the NIDS-Vis framework.

TABLE III
PERFORMANCE POSITIONING: HFS-VAE VS. NIDS-VIS.

Model	Dataset	AP	F1	AUC
NIDS-Vis [6]	UQ-IoT-IDS	0.802	0.69	0.84
HFS-VAE (Ours)	CICIDS2017	0.841	0.74	0.88
HFS-VAE (Ours)	NSL-KDD	0.923	0.73	0.88

While Table III shows that HFS-VAE records higher AP, F1, and AUC, These results are not directly comparable since the datasets differ: NIDS-Vis was evaluated on UQ-IoT, whereas HFS-VAE is tested on CICIDS2017 and NSL-KDD. Therefore, Table III should be interpreted as a contextual positioning rather than a strict numerical benchmark. Its inclusion highlights how HFS-VAE extends the feature-partitioning concept of NIDS-Vis from constrained IoT settings to enterprise-scale environments.

H. Practical Considerations

Beyond numerical performance, practical aspects of HFS-VAE were also considered. Training requires moderate computational resources (approximately 100 epochs on a single RTX 3060 GPU). But inference is lightweight: a single flow can be classified in under 5 ms, making the system suitable for real-time detection scenarios. The model's modular design

allows efficient scaling across parallel encoders, while memory requirements remain modest (under 200 MB during inference). These characteristics suggest that HFS-VAE can be feasibly deployed in enterprise monitoring pipelines, With further optimizations, such as pruning or quantization,n enables adaptation to IoT and edge devices.

HFS-VAE achieves very high precision and clear latent separation, as confirmed by PCA and MLE. Its recall remains limited at 0.4332 under the optimal threshold, reflecting a precision–recall trade-off. Even so, it delivers strong detection performance, effective anomaly isolation, and improved interpretability for enterprise-scale use.

VII. Conclusion and Future Work

This study presents HFS-VAE, a hierarchical feature-splitting variational autoencoder that improves robustness and interpretability in anomaly-based intrusion detection. The model extends NIDS-Vis by combining feature grouping, multi-branch encoders, and classifier fusion for accurate anomaly localisation and transparent latent analysis.

On CICIDS2017 and NSL-KDD, HFS-VAE achieved F1-scores of 0.7463 and 0.7371, AP values of 0.8412 and 0.9234, and AUC values near 0.88. Visualization techniques (PCA, MLE, boundary analysis) confirmed effective feature disentanglement.

Compared with AE, VAE, SOVAE, and NIDS-Vis, the model shows stronger anomaly separation and modularity. Its main drawback is low recall (0.4332 at the optimal threshold), reflecting a precision–recall trade-off that future work must address.

A. Future Work

Future directions include:

- Extending HFS-VAE to streaming or online environments for real-time detection.
- Optimizing for IoT/edge devices through pruning and quantization.
- Exploring adversarial training and threshold-adaptive methods to strengthen resilience and improve recall.
- Leveraging latent embeddings for fine-grained attack categorization.
- Expanding evaluation to additional datasets beyond CICIDS2017 and NSL-KDD for broader validation.

In summary, HFS-VAE provides a structured, interpretable, and adaptable approach that bridges academic research with practical cybersecurity applications, while also highlighting open challenges such as recall optimization and broader validation.

References

[1] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 2, pp. 41–50, 2018.

[2] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019.

[3] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel, "Deep learning for anomaly detection: A review," *ACM Comput. Surv.*, vol. 54, pp. 1–38, 2021.

[4] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv:1312.6114*, 2013.

[5] X. Wang, P. Liu, and Y. Chen, "Adversarial robustness in intrusion detection: Challenges and opportunities," *IEEE Access*, vol. 9, pp. 123 755–123 770, 2021.

[6] K. He, D. D. Kim, and M. R. Asghar, "Nids-vis: Improving the generalized adversarial robustness of network intrusion detection system," *Comput. Secur.*, vol. 145, p. 104028, 2024.

[7] J. Marcelino, J. Abawajy, and A. Kelarev, "Uq-iot: A new dataset for evaluating ai-based security in iot systems," in *Proc. Int. Conf. Comput. Sci. Appl. (ICCSA)*, 2021, pp. 311–326.

[8] N. Quadri, S. Yasmeen, K. Charan, K. S. Babu, and S. Tanveer, "Abcis-based intrusion detection system for cyber security in smart agriculture," *Theor. Appl. Inf. Technol.*, vol. 102, pp. 610–618, 2024.

[9] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[10] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," *arXiv:1802.03426*, 2018.

[11] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.

[12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116.

[13] Y. Chen and T. Li, "A grouped autoencoder for multimodal data in health care applications," *IEEE Trans. Biomed. Health Inform.*, vol. 25, pp. 877–881, 2021.

[14] Y. Liu, J. Zhang, and M. Yu, "An intrusion detection system based on xgboost," in *Proc. IEEE Int. Conf. Smart Internet Things (SmartIoT)*, 2022, pp. 72–76.

[15] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[16] M. Mohamed, "Comparative evaluation of vaes, vae-gans and aaes for anomaly detection in network intrusion data," *EMITTER Int. J. Eng. Technol.*, vol. 11, pp. 160–173, 2023.

[17] Y. Ren, K. Feng, F. Hu, L. Chen, and Y. Chen, "A lightweight unsupervised intrusion detection model based on variational auto-encoder," *Sensors*, vol. 23, p. 8407, 2023.

[18] M. Farhan, H. W. U. Din, S. Ullah, M. S. Hussain, M. A. Khan, T. Mazhar, U. F. Khattak, and I. H. Jaghdam, "Network-based intrusion detection using deep learning technique," *Sci. Rep.*, vol. 15, p. 25550, 2025.

[19] S. Jamshidi, A. Nikanjam, K. W. Nafi, F. Khomh, and R. Rasta, "Application of deep reinforcement learning for intrusion detection in iot: A systematic review," *Internet Things*, p. 101531, 2025.

[20] Y. Zhang, R. C. Muniyandi, and F. Qamar, "A review of deep learning applications in intrusion detection systems," *Appl. Sci.*, vol. 15, p. 1552, 2025.

[21] Y. Xue, C. Kang, and H. Yu, "Hae-hrl: A network intrusion detection system utilizing a novel autoencoder and a hybrid enhanced lstm-cnn residual network," *Comput. Secur.*, vol. 151, p. 104328, 2025.

[22] A. Kumar, R. Rajamani, M. Sumithra, P. Kaliyaperumal, B. Balusamy, and F. Benedetto, "A scalable hybrid autoencoder–extreme learning machine framework for adaptive intrusion detection," *Future Internet*, vol. 17, p. 221, 2025.

[23] L. Mhamdi and M. M. Isa, "Securing sdn: Hybrid autoencoder-random forest for intrusion detection and attack mitigation," *J. Netw. Comput. Appl.*, vol. 225, p. 103868, 2024.

[24] N. Saranya and A. Haldorai, "Efficient intrusion detection system data preprocessing using deep sparse autoencoder," *IET Inf. Secur.*, vol. 2024, p. 9937803, 2024.

[25] Z. Li, C. Huang, and W. Qiu, "An intrusion detection method combining variational auto-encoder and generative adversarial networks," *Comput. Netw.*, vol. 253, p. 110724, 2024.

[26] H. Huang, J. Yang, H. Zeng, Y. Wang, and L. Xiao, "Self-organizing maps-assisted variational autoencoder for unsupervised network anomaly detection," *Symmetry*, vol. 17, p. 520, 2025.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

8% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 29 Not Cited or Quoted 8%
  Matches with neither in-text citation nor quotation marks
- 2 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

4% Internet sources
5% Publications
6% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.